



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

ANALIZA PARADIGMELOR DE ANOMALY DETECTION PE SETURI DE DATE CU STRUCTURĂ TEMPORALĂ DIFERITĂ

Absolvent

Savin Radu-Andrei

Coordonator științific

Paul Irofti

București, septembrie 2026

Rezumat

Această lucrare investighează în ce măsură modul în care un model de detecție a anomaliilor reprezintă structura latentă a datelor „normale” determină tipurile de anomalii pe care le poate detecta. Șapte modele, acoperind patru paradigme structural distincte (graniță, partiționare, bottleneck de compresie, obiectiv predictiv), sunt evaluate printr-un pipeline uniform pe trei seturi de date cu grade diferite de structură temporală – tranzacții bancare tabelare, serii de timp Yahoo Webscope S5 și fluxuri de rețea CIC-IDS2017. Rezultatele arată că performanța fiecărei paradigme depinde sistematic de tipul de anomalie și de structura datelor, nu de puterea intrinsecă a modelului, iar o serie de studii de caz de explicabilitate confirmă direct mecanismele propuse pentru a explica aceste diferențe. Un ansamblu construit pe baza a două paradigme cu evidențe demonstrabil disjuncte extinde acoperirea pe clase de atac altfel nedetectabile individual, confirmând valoarea combinării unor asumptii complementare despre normalitate.

Abstract

This thesis investigates the extent to which the way an anomaly detection model represents the latent structure of „normal” data determines the types of anomalies it can detect. Seven models, spanning four structurally distinct paradigms (boundary, partitioning, compression bottleneck, predictive objective), are evaluated under a uniform pipeline across three datasets with varying degrees of temporal structure – tabular credit card transactions, Yahoo Webscope S5 time series, and CIC-IDS2017 network flows. Results show that each paradigm’s performance depends systematically on the type of anomaly and the structure of the data, rather than on the model’s intrinsic strength, and a series of explainability case studies directly confirm the mechanisms proposed to explain these differences. An ensemble built from two paradigms with demonstrably disjoint evidence extends coverage to attack classes otherwise undetectable individually, confirming the value of combining complementary assumptions about normality.

Cuprins

1	Introducere	6
1.1	Context și Motivație	6
1.2	Structura Lucrării	6
2	Preliminarii	7
2.1	Proiectarea și Abordarea Cercetării	7
2.1.1	Ipoteze și Comportament Așteptat	7
2.2	Algoritmi de Detecție a Anomaliilor	8
2.2.1	One-Class SVM	8
2.2.2	Isolation Forest	8
2.2.3	Autoencoder Feedforward (MLP)	8
2.2.4	Autoencoder LSTM	9
2.2.5	LSTM Predictiv	9
2.2.6	Autoencodere Bazate pe Transformer	9
2.3	Seturile de Date	9
2.3.1	Credit Card Fraud Detection (Setul de Date A)	9
2.3.2	Yahoo Webscope S5 (Setul de Date B)	10
2.3.3	CIC-IDS2017 (Setul de Date C)	10
3	Metode și Configurare Experimentală	11
3.1	Preprocesare	11
3.1.1	Preprocesare Tabelară (Credit Card)	11
3.1.2	Crearea de Ferestre pentru Serii de Timp (Yahoo S5)	11
3.1.3	Preprocesarea Fluxurilor de Rețea (CIC-IDS2017)	12
3.2	Protocolul de Evaluare	12
3.2.1	Metrici Raportate	12
3.2.2	Selecția Pragului	12
3.2.3	Fără Protocolul <i>Point Adjustment</i>	13
4	Rezultate Experimentale	13
4.1	Detectarea Fraudei cu Cardul de Credit (Setul de Date A)	13
4.1.1	Comparație Generală	13
4.1.2	Discuție	14
4.2	Yahoo Webscope S5 (Setul de Date B)	14
4.2.1	A1 – Anomalii Punctuale (Metrici Reale de Server)	15
4.2.2	A3 – Anomalii Contextuale/Sezoniere	15
4.2.3	A4 – Anomalii de Tip Schimbare-de-Nivel	16

4.2.4	Verificare de Referință Trivială (Doar Yahoo)	16
4.2.5	Discuție	16
4.3	CIC-IDS2017 (Setul de Date C)	17
4.3.1	Comparație Generală	17
4.3.2	Detaliere pe Tip de Atac	18
4.3.3	Discuție	19
5	Studii de Caz de Explicabilitate	20
5.1	Localizarea Erorii de Reconstrucție pe Yahoo A3	20
5.1.1	Verificare Cantitativă: Coeficientul Gini al Concentrării Erorii	20
5.2	Regimuri de Caracteristici Disjuncte: AE+Context vs. Temporal IForest	22
5.2.1	Temporal IForest: Atribuire SHAP pe Toată Fereastra	22
5.2.2	AE+Context: SHAP Dominat de Caracteristici de Context al Gazdei	23
5.2.3	Construirea Scorului Combinat	24
5.2.4	Acoperire per Clasă vs. AP Agregat	25
6	Concluzii	25
	Anexe	26
1	Detalii ale Mecanismelor de Model	26
1.1	One-Class SVM	26
1.2	Isolation Forest	26
1.3	Autoencodere (MLP, LSTM) și LSTM Predictiv	26
1.4	Autoencodere Bazate pe Transformer	27
2	Detalii Suplimentare de Preprocesare	27
2.1	Preprocesare Tabelară (Credit Card)	27
2.2	Crearea de Ferestre pentru Serii de Timp (Yahoo S5)	28
2.3	Preprocesarea Fluxurilor de Rețea (CIC-IDS2017)	29
3	Alegeri legate de Hiperparametri și Antrenare	30
3.1	Isolation Forest	30
3.2	One-Class SVM	31
3.3	Autoencoder Feedforward (MLP)	31
3.4	LSTM Predictiv	31
3.5	Autoencoder LSTM	32
3.6	Bottleneck Transformer Autoencoder	32
3.7	Causal Transformer	32
4	Protocolul de Evaluare: Metrici și Rezultate Complete	33
4.1	Metrici Raportate	33

4.2	Rezultate Complete – Credit Card Fraud Detection	34
4.3	Rezultate Complete – Yahoo Webscope S5	34
4.4	Rezultate Complete – CIC-IDS2017	34
5	Verificare de Referință Trivială – Yahoo S5 (Detalii)	35
6	Compoziția pe Clase de Atac a Setului de Test CIC-IDS2017	35
7	Hărți Termice ale Erorii de Reconstrucție – Yahoo A3	37
8	Validarea Scorului Temporal IForest Sub Etichetare pe Ultimul Flux	38
9	Diagnostic SHAP Suplimentare	38
10	Alinierea Scorurilor și Eșecul Standardizării z pentru Ansamblul AE+Context + Temporal IForest	40
	Bibliografie	40

Capitolul 1

Introducere

1.1 Context și Motivație

Această lucrare este un studiu experimental în învățarea automată, mai precis în subdomeniul *deteției de anomalii* (*anomaly detection*) – sarcina de a detecta puncte, evenimente sau structuri care deviază de la normalitate. Domeniul este unul larg și acoperă o varietate de probleme, dintre care în această lucrare sunt abordate: detecția fraudelor în tranzacții bancare, detecția atacurilor asupra rețelelor de calculatoare și detecția defecțiunilor în metricile monitorizate ale serverelor.

O primă distincție este cea dintre *outlier detection*, care antrenează pe date ce conțin deja anomalii, și *novelty detection*, care antrenează exclusiv pe date normale și evaluează, la testare, capacitatea modelului de a detecta devieri de la normalitatea astfel învățată. Această lucrare adoptă abordarea *novelty detection*, cadrul folosit practic în cazurile de utilizare de mai sus (un model de detecție a fraudelor nu este antrenat pe cazuri confirmate de fraudă), care ajută și la formularea scopului lucrării printr-o singură întrebare: în ce măsură modul în care un model reprezintă structura latentă a datelor „normale” determină tipurile de anomalii pe care le poate detecta?

Motivația, cât și scopul acestei lucrări sunt, așadar, căutarea unui răspuns la această întrebare și abordarea ulterioară a ipotezelor formate în urma rezultatelor experimentale.

Contribuția proprie a acestei lucrări constă, așadar, în:

1. un pipeline standardizat de preprocesare, antrenare și evaluare, aplicat uniform pe trei seturi de date diferite și pe până la treisprezece configurații de modele și seturi de caracteristici (*features*);
2. o analiză sistematică a performanței diferitelor paradigme în funcție de tipul setului de date; și
3. o serie de studii de caz de explicabilitate, menite să testeze direct ipoteze formulate în secțiunea de discuții a rezultatelor, folosind hărți termice ale erorii de reconstrucție, o măsură de concentrație bazată pe coeficientul Gini și valori de atribuire SHAP, în locul unor concluzii bazate exclusiv pe metrici agregate.

1.2 Structura Lucrării

Restul lucrării este organizat în cinci capitole: Capitolul 2 (*Preliminarii*) introduce fundalul teoretic – funcționarea și paradigma fiecărui model testat, și seturile de date folosite;

Capitolul 3 (*Metode și Configurare Experimentală*) descrie pipeline-urile de preprocesare, alegerile de hiperparametri/arhitectură și protocolul de evaluare; Capitolul 4 (*Rezultate Experimentale*) prezintă rezultatele pe seturi de date, cu discuții care le raportează la așteptările din Preliminarii; Capitolul 5 (*Studii de Caz de Explicabilitate*) revizitează și testează direct, prin diagnostice țintite, ipoteze specifice ridicate în discuția Capitolului 4, confirmând unele și nuanțând altele; iar Capitolul 6 (*Concluzii*) rezumă concluziile, discută limitările și schițează direcții de dezvoltare ulterioară.

Capitolul 2

Preliminarii

2.1 Proiectarea și Abordarea Cercetării

Pentru a răspunde la întrebarea formulată mai sus, în loc să optimizez o singură arhitectură pentru un singur set de date, am realizat o comparație de-a lungul a două axe simultan.

Prima axă este **paradigma**: cele șapte modele evaluate (§2.2) acoperă patru moduri structural distincte de a reprezenta cum arată „normalul” – o **graniță** trasată într-un spațiu indus de un kernel (One-Class SVM), o **partiționare** a spațiului de caracteristici prin diviziuni aleatoare (Isolation Forest), un **bottleneck de compresie** învățat, care trebuie să renunțe la orice, cu excepția structurii dominante a datelor normale (cele trei autoencodere bazate pe reconstrucție), și un model **predictiv** al evoluției pas cu pas a datelor normale (LSTM predictiv, Transformer cauzal). Fiecare paradigmă codifică o presupunere implicită diferită despre ce anume face un punct anormal, ceea ce ridică întrebarea centrală a acestei lucrări: care presupunere se potrivește cărui tip de anomalie.

A doua axă este **setul de date**: Credit Card Fraud, Yahoo Webscope S5 și CIC-IDS2017 (§2.3) acoperă un spectru de structură temporală – de la un set pur tabelar, fără ordonare semnificativă între rânduri, trecând prin serii de timp cu adevărat secvențiale, până la fluxuri de rețea care admit atât o perspectivă plată, cât și una temporală asupra acelorași evenimente. Rularea fiecărei paradigme pe fiecare punct al acestui spectru face posibilă atribuirea unui rezultat dat *paradigmei*.

2.1.1 Ipoteze și Comportament Așteptat

Pe baza acestor două axe, anticipez două tipare în rezultate.

În primul rând, mă aștept ca paradigmele care oferă modelului o structură temporală inherentă a datelor – prin construcție, nu doar prin faptul că le sunt oferite ferestre de intrare – să performeze în general mai bine pe seturile de date cu adevărat secvențiale (Yahoo S5 și, într-o măsură mai mică, varianta temporală a CIC-IDS2017), comparativ cu paradigmele care tratează fiecare eșantion ca independent.

În al doilea rând, mă aștept ca diferite paradigme să fie mai bine adaptate la a detecta diferite *tipuri* de anomalii, chiar în cadrul aceluiași set de date. Dacă această presupunere este corectă, atunci compunerea mai multor paradigme complementare într-un ansamblu ar trebui să ducă la o îmbunătățire a performanței față de oricare model individual, atâta timp cât modelele componente exploatează relații diferite din date, și nu doar variații ale aceleiași relații. Această a doua ipoteză este testată direct atât prin analiza pe tip de atac din §4 (în special pentru CIC-IDS2017, unde este disponibilă eticheta tipului de atac), cât și prin studiul de caz al modelului ansamblu din §5.

În următoarele secțiuni vor fi prezentate detaliile algoritmilor și seturilor de date alese.

2.2 Algoritmi de Detecție a Anomaliilor

2.2.1 One-Class SVM

One-Class SVM (formularea ν -SVM) învață o graniță în jurul datelor de antrenare, într-un spațiu de caracteristici indus de un kernel. Trei hiperparametri, fixați identic pe toate cele trei seturi de date (`kernel='rbf'`, `nu=0.01`, `gamma='scale'` – vezi §3), determină împreună forma acestei granițe (detalii despre rolul fiecăruia: Anexa 1). La testare, distanța semnată față de graniță (pozitivă în interior, negativă în exterior) este negată astfel încât, în concordanță cu toate celelalte modele din acest proiect, *scor mai mare = mai anormal*.

2.2.2 Isolation Forest

Isolation Forest construiește un ansamblu de arbori care împart fiecare recursiv spațiul de caracteristici pe baza unor caracteristici și puncte de divizare alese aleator, folosind lungimea medie a căii necesare pentru a izola un punct drept semnal de anomalie: punctele din regiuni rare de densitate scăzută (anomaliile) se izolează în mai puține diviziuni, deci ating o lungime medie a căii mai mică decât punctele benigne tipice. Scorul returnat este această lungime medie a căii, negată, pentru convenția consistentă *scor mai mare = mai anormal*. Parametrul `contamination` este lăsat la valoarea implicită `'auto'` și nu are niciun efect asupra rezultatelor, întrucât datele de antrenare sunt exclusiv benigne, iar pragul de operare este ales extern prin baleierea curbei PR (§3.2) – detalii: Anexa 1.

2.2.3 Autoencoder Feedforward (MLP)

Autoencoder Feedforward este bazat pe reconstrucție: o pereche simetrică encoder/decoder de straturi `Linear` + `ReLU` comprimă vectorul de intrare printr-un bottleneck de dimensionalitate redusă, apoi îl reconstruiește, fiind antrenat să minimizeze eroarea medie pătratică (MSE) de reconstrucție. Bottleneck-ul mai îngust decât intrarea forțează o reprezentare comprimată a structurii dominante *benigne*, în loc de funcția identitate. La testare, scorul de anomalie

este MSE per eşantion: eşantioanele benigne se reconstruiesc cu eroare mică, cele anormale – pentru care bottleneck-ul nu a fost antrenat – se reconstruiesc slab.

2.2.4 Autoencoder LSTM

Autoencoder LSTM aplică acelaşi principiu de reconstrucţie, dar pentru secvenţe: un strat LSTM encoder consumă întreaga secvenţă şi păstrează doar starea ascunsă finală (şi starea de celulă) ca vector sumar de dimensiune fixă, care este apoi repetată la fiecare pas ca intrare pentru un LSTM decoder, ale cărui ieşiri sunt proiectate înapoi la dimensionalitatea originală. Scorul de anomalie este, din nou, MSE per eşantion, calculat pe toţi paşii şi toate caracteristicile simultan.

2.2.5 LSTM Predictiv

LSTM Predictiv foloseşte acelaşi bloc recurent, dar cu un obiectiv diferit: este antrenat să prezică pasul $t + 1$ din paşii $0..t$, la fiecare poziţie simultan – fără separare encoder/decoder şi fără bottleneck, starea ascunsă la fiecare pas fiind proiectată direct într-o predicţie prin un strat liniar. Scorul de anomalie este MSE dintre valorile prezise şi cele reale ale pasului următor.

2.2.6 Autoencodere Bazate pe Transformer

Sunt folosite două variante, ambele înlocuind coloana vertebrală recurentă cu un **TransformerEncoder** (blocuri de auto-atenţie + feedforward, codificare poziţională sinusoidală): **Bottleneck Transformer AE**, bazat pe reconstrucţie – atenţie non-cauzală urmată de mediere globală (*global-average-pooling*) într-un vector comprimat printr-un bottleneck liniar (analog stării ascunse din LSTM AE), apoi extins înapoi; scor: MSE de reconstrucţie – şi **Causal Transformer**, predictiv, echivalentul bazat pe atenţie al LSTM Predictiv (mască de atenţie cauzală, fără bottleneck; scor: MSE de predicţie). Detalii mecanism: Anexa 1.

Ambele variante folosesc deliberat un singur strat **TransformerEncoder**, păstrând costul de antrenare/inferenţă şi numărul de parametri la aproximativ jumătate din ce ar necesita un design encoder-decoder echivalent, fără a sacrifica abilitatea de a face fie comprimare-şi-reconstrucţie, fie predicţie cauzală. Nu există o implementare Causal Transformer pentru setul de date credit card (§4).

2.3 Seturile de Date

2.3.1 Credit Card Fraud Detection (Setul de Date A)

Setul de date Kaggle Credit Card Fraud Detection conţine 284,807 tranzacţii bancare, dintre care 492 (0.172%) sunt frauduloase. Cele 30 de caracteristici prezente sunt: **Time** (secunde scurse de la prima tranzacţie din set), 28 de caracteristici transformate prin PCA

V1–V28 (anonimizate din motive de confidențialitate, deja decorelate și aproximativ zero-medie prin construcție), și **Amount** (valoarea brută a tranzacției, puternic asimetrică spre dreapta). **Time** reprezintă secunde scurse, nu o axă temporală structurată – nu există periodicitate, tendință sau noțiune de sesiune de exploatat. Acest lucru face din Setul de Date A cazul de control deliberat, „fără ordine temporală reală”.

Setul de date este disponibil pe Kaggle [1], alături de un notebook exploratoriu dedicat gestionării datelor dezechilibrate [2], care a servit drept inspirație pentru o parte din scripturile acestui proiect.

2.3.2 Yahoo Webscope S5 (Setul de Date B)

Yahoo Webscope S5 constă din patru familii de benchmark (A1–A4), fiecare o colecție de mai multe serii de timp univariate independente, cu anomalii etichetate, dar fiecare construită pentru a izola un tip diferit de anomalie:

Benchmark	Serii	Origine	Tip de anomalie	Rata de anomalii
A1	67	Metrii reale de server	Valori punctuale aberante	1.76%
A2	100	Sintetic, o singură componentă periodică + tendință	Valori punctuale aberante	0.33%
A3	100	Sintetic, 3 componente de frecvență în interacțiune	Contextuală (deviație de fază)	0.56%
A4	100	Sintetic, aceeași structură, fără sezonabilitate	Schimbări de nivel	0.50%

A1 este real și zgomotos (cicluri zilnice/săptămânale dezordonate); A2–A4 sunt sintetice și construite special astfel încât tipul de anomalie să fie controlat independent de forma seriei de bază. Acest lucru este ceea ce permite capitolului de Rezultate să atribuie diferențele de performanță *tipului de anomalie* (punctuală vs. contextuală vs. schimbare de nivel), și nu unor diferențe confundante între seturile de date.

2.3.3 CIC-IDS2017 (Setul de Date C)

CIC-IDS2017 (Canadian Institute for Cybersecurity) este un benchmark de detecție a intruziunilor în rețea, captat pe parcursul a cinci zile: ~2.8 milioane de fluxuri de rețea, fiecare descris de ~80 de caracteristici statistice extrase prin CICFlowMeter (durată, numărul și rata pachetelor/octeților, statistici ale timpului dintre sosiri, numărul de flag-uri TCP, dimensiuni de fereastră etc.). Luni este trafic 100% benign; 14 tipuri distincte de atacuri (variante DoS/DDoS, PortScan, forță brută, atacuri web, Bot, Infiltration, Heartbleed) sunt injectate de marți până vineri.

Se știe că distribuția brută/Kaggle a acestui set de date este substanțial coruptă [3]: aproximativ 26% din fluxuri sunt artefacte de tip „appendix” TCP, care moștenesc eticheta de atac a fluxului părinte fără a conține trafic de atac real; o mare parte din fluxurile etichetate nominal ca atac nu au încărcătură utilă reală (etichetate doar formal, nu pe bază de comportament); iar clasa DoS **Hulk** corespunde unei simulări de atac defectă, care nu se execută niciodată efectiv împotriva serverului țintă. Coloanele de identificare prezente (**Flow ID**, **IP sursă/destinație**, **Timestamp**) ar permite unui model să „învețe pe scurtătură” după adresa IP sau fereastra de captură. Acest proiect folosește versiunea curățată și reextrasă de la Engelen,

Rimmer, Joosen [3], în locul CSV-urilor originale Kaggle/UNB, și elimină explicit coloanele de identificare (§3.1).

Fluxurile CIC-IDS2017 sunt agregate statistice la nivel de sesiune. Acest lucru face din acest set de date un caz de tranziție perfect: admite o perspectivă validă de tip plat/tabelar (un flux = un eșantion, analog setului de Date A), o perspectivă temporală validă (fluxuri grupate și ordonate per gazdă, analog setului de Date B).

Capitolul 3

Metode și Configurare Experimentală

Hiperparametrii și alegerile legate de antrenare, per model (optimizator, epoci, lățimi de straturi și restul detaliilor de arhitectură, câte un tabel per model pentru cele patru variante de set de date/caracteristici) sunt prezentate în Anexa 3.

3.1 Preprocesare

Două principii se aplică la toate cele trei pipeline-uri: (1) fiecare model este antrenat exclusiv pe eșantioane benigne/normale (*novelty detection*) – partiția de antrenare nu conține niciodată o anomalie, iar scalarele/normalizatoarele sunt *potrivite* doar pe antrenare, apoi aplicate neschimbate pe testare, evitând data leakage; (2) partiția de testare conține întotdeauna ambele clase, evaluate cu protocolul din §3.2. Multe dintre detaliile considerate neesențiale pentru înțelegerea textului au fost mutate în Anexa 2.

3.1.1 Preprocesare Tabelară (Credit Card)

Time este eliminat (nu reprezintă informație semnificativă). Rândurile benigne (**Class=0**) sunt împărțite 80/20 în antrenare/test-benign, iar setul de testare adaugă *toate* cele 492 de rânduri de fraudă. Scalarea este asimetrică: **V1--V28** rămân nescalate (deja caracteristici PCA), iar **Amount** este scalat cu un **RobustScaler**, ales pentru robustețea sa la coada dreaptă accentuată. Pași exacți: Anexa 2.

3.1.2 Crearea de Ferestre pentru Serii de Timp (Yahoo S5)

Fiecare serie este împărțită temporal 70/30 (**TRAIN_FRAC=0.70**, fără amestecare aleatoare, pentru a nu lăsa informație din viitor să se scurgă în antrenare), scalată *per serie* cu un **MinMaxScaler** potrivit doar pe punctele neanormale de antrenare, apoi transformată în ferestre de lungime **WINDOW=64** (pas 4 la antrenare, pas 1 la testare). Eticheta unei ferestre urmează regula „oricare”: pozitivă dacă cel puțin un pas este anormal. Detalii complete, plus comparația metodologică cu referința DeepAnt [4]: Anexa 2.

3.1.3 Preprocesarea Fluxurilor de Rețea (CIC-IDS2017)

Există trei implementări, toate având la bază aceiași pași de curățare (eliminarea coloanelor de identificare, codificare one-hot pentru Protocol, imputare cu mediana, MinMaxScaler potrivit pe antrenare, împărțire pe zile: luni antrenare, marți–vineri testare), dar diferind prin unitatea de observație: **Plat** (un rând = un flux, 77 caracteristici), **Temporal** (un rând = o fereastră de WINDOW=16 fluxuri consecutive de la aceeași adresă IP sursă) și **Context de gazdă** (fluxuri individuale suplimentate cu 9 caracteristici cauzale dintr-o fereastră glisantă de 60s per IP gazdă – număr de fluxuri, porturi unice, rata de diversitate a porturilor etc. – urmând adaptarea nesupervizată a lui Raskovalov, Gabdullin, Androsov [5], 86 caracteristici în total). Gruparea se face după IP sursă, întrucât comportamentul la nivel de gazdă caracterizează atacuri precum PortScan sau DoS mai bine decât o singură conexiune. Detalii complete: Anexa 2.

3.2 Protocolul de Evaluare

La testare, fiecare model produce un scor scalar de anomalie per eșantion – niciodată o etichetă directă – iar această etichetă este obținută prin aplicarea unui prag asupra scorului, urmând procedura descrisă mai jos.

3.2.1 Metrici Raportate

Lista completă a metricilor raportate de fiecare rulare, precum și rezultatele complete pe toate aceste metrice pentru fiecare model și set de date, sunt prezentate în Anexa 4.

PR-AUC (Precizia Medie) este tratată drept metrica principală pentru compararea modelelor, în locul F1 sau ROC-AUC, urmând raționamentul lui Kim et al. [6]: prestabilirea unui prag de decizie fără acces la setul de testare nu este practică, iar metodele existente raportează în schimb F1 la orice prag care se întâmplă să îl maximizeze, ceea ce face rezultatele dependente de prag prin construcție. Aceștia concluzionează că „metrice suplimentare cu dependență redusă, precum AUROC sau aria de sub curba precizie-recall (AUPR), vor ajuta la o evaluare riguroasă” – justificarea adoptată aici pentru clasificarea modelelor după PR-AUC.

ROC-AUC este de asemenea raportat, mai mult pentru completitudine/comparabilitate cu literatura anterioară.

Pentru CIC-IDS2017, este raportată suplimentar Precizia Medie defalcată pe tip de atac, pentru a susține discuția per-tip-de-atac din §4.

3.2.2 Selecția Pragului

Se alege un singur prag de decizie per model/set de date pentru metricile secundare, dependente de prag (F1, acuratețe echilibrată, matrice de confuzie): se baleiază toate pragurile de pe curba precizie-recall și se alege cel care maximizează F_β cu $\beta = 2$, ponderând recall-

ul dublu față de precizie. Valoarea lui β nu este reglată – orice $\beta > 1$ ar exprima aceeași preferință operațională rezonabilă, unde o anomalie ratată e de obicei mai costisitoare decât o alarmă falsă; $\beta = 2$ e o alegere reprezentativă.

Acesta este un prag de tip *oracol*, selectat folosind chiar curba precizie-recall a setului de testare, ceea ce nu ar fi mereu disponibil în realitate. Este acceptabil aici întrucât obiectul de studiu este comparația relativă între modele la cel mai bun punct de operare, iar PR-AUC nu depinde de selecția pragului – cifrele F2/F1/acuratețe echilibrată la pragul oracol rămân secundare, ilustrative.

3.2.3 Fără Protocolul *Point Adjustment*

Multe lucrări despre detecția anomaliilor în serii de timp aplică *Ajustarea de Punct (Point Adjustment, PA)*: dacă orice punct dintr-un segment contiguu de anomalie este semnalat, întregul segment e considerat detectat. Kim et al. [6] arată că aceasta poate umfla F1 până aproape de 1 chiar și pentru scoruri aleatorii, atunci când segmentele de anomalie sunt lungi, iar F1 ajustat prin PA este aproape necorelat cu F1 neajustat (Pearson $r = -0.59$ pe SWaT, în studiul lor). Nicio ajustare PA nu este aplicată nicăieri în acest proiect: etichetele sunt fixate la preprocesare, iar predicțiile sunt evaluate exact așa cum rezultă din pragul aplicat, fără ajustare retroactivă.

Capitolul 4

Rezultate Experimentale

4.1 Detectarea Fraudei cu Cardul de Credit (Setul de Date A)

4.1.1 Comparație Generală

Model	Paradigmă	ROC-AUC	Precizie Medie	F1 (fraudă)
MLP Autoencoder	Reconstrucție	0.9649	0.7105	0.6992
Isolation Forest	Partiționare	0.9587	0.6725	0.6070
LSTM Autoencoder	Reconstrucție (pseudo-secv.)	0.9476	0.6223	0.7362
OC-SVM	Frontieră	0.9354	0.5959	0.6454
Transformer Bottleneck AE	Reconstrucție (pseudo-secv.)	0.9583	0.5947	0.6137
Predictive LSTM	Predictiv (pseudo-secv.)	0.9581	0.5190	0.5244

Tabela 4.1: Detectarea fraudei cu cardul de credit – comparație de modele, clasificate după Precizia Medie.

Toate cele șase modele ating ROC-AUC > 0.93 , dar AP variază între 0.519–0.711 $\rightarrow$ ROC-AUC este mult mai puțin sensibil la dezechilibrul de clasă față de AP.

4.1.2 Discuție

Nu există o secvență autentică în acest set de date. Spre deosebire de Yahoo S5 sau variantele temporale/contextuale ale CIC-IDS2017, rândurile de tranzacții nu au nicio ordine temporală sau cauzală între caracteristici: **Time** este eliminată la preprocesare (§3.1); modelele secvențiale au fost adăugate ulterior exclusiv pentru comparabilitate între seturile de date. Cele trei arhitecturi „temporale” evaluate aici (Predictive LSTM, LSTM AE, Transformer Bottleneck AE) nu văd o secvență reală – ele reformează fiecare rând cu 29 de caracteristici într-o secvență artificială de 29 de pași scalari ($((N, 29) \rightarrow (N, 29, 1))$), câte o caracteristică pe pas, în ordinea din fișierul .csv.

Impact asupra rezultatelor. Predictive LSTM este cel mai slab model (AP 0.519): obiectivul său de predicție a următorului pas presupune o relație cauzală între pași consecutivi, dar componentele PCA adiacente sunt ortogonale/decorelate prin construcție, deci nu există niciun semnal real de învățat – reflectat în cel mai slab număr de fals-pozitive (FP=566). LSTM AE se descurcă comparativ bine (AP 0.622, cel mai bun F1 dintre toate modelele, 0.736, cel mai mic FP, 174), probabil întrucât encoderul său comprimă întreaga intrare într-o singură stare ascunsă sumară, funcționând similar bottleneck-ului MLP AE, mai degrabă decât depinzând doar de ordine. Transformer Bottleneck AE (AP 0.595) se situează aproape de modelele clasice – apare agregarea medie globală (global average pooling) într-un vector latent. MLP Autoencoder, singura arhitectură care tratează cele 29 de caracteristici ca vector plat obișnuit, câștigă per total (AP 0.7105) – consistent cu absența oricărei secvențe de exploatat. Impunerea unei structuri secvențiale acolo unde nu există niciuna pare să fie un handicap.

Observații la nivel de paradigmă. Reconstrucția este cea mai puternică paradigmă per total, odată ce se ia în calcul handicapul pseudo-secvențial (MLP AE, LSTM AE și Transformer Bottleneck AE ocupă trei din primele patru locuri). Isolation Forest este cel mai bun model din afara paradigmei de reconstrucție (AP 0.6725) și al doilea per total. OC-SVM (AP 0.5959) se situează la mijloc, cu al doilea cel mai mare număr de fals-pozitive după Predictive LSTM, singurul reprezentant al paradigmei predictive aici și cel mai slab model per total, din motivul structural discutat mai sus.

Sanity check. F1-ul cel mai bun dintre toate (0.7362) al LSTM AE, în ciuda unui AP mediu (0.6223), reamintește că F1 este calculat la un singur prag oracol F2 și poate favoriza un punct de operare diferit de cel care maximizează aria de sub curba PR.

4.2 Yahoo Webscope S5 (Setul de Date B)

A2 este exclus din comparațiile de mai jos: la fel ca A1, conține doar anomalii punctuale, fără a adăuga un tip nou de anomalie în comparație.

4.2.1 A1 – Anomalii Punctuale (Metrici Reale de Server)

Causal Transformer și Transformer Bottleneck AE nu au fost rulate pe acest benchmark.

Model	Paradigmă	ROC-AUC	Precizie Medie
MLP Autoencoder	Reconstrucție	0.792	0.611
LSTM Autoencoder	Reconstrucție	0.794	0.600
OC-SVM	Frontieră	0.776	0.595
Predictive LSTM	Predictiv	0.789	0.588
Isolation Forest	Partiționare	0.765	0.511

Tabela 4.2: Yahoo S5 A1 (anomalii punctuale) – comparație de modele, clasificate după Precizia Medie.

Discuție. Toate cele cinci modele se grupează strâns în intervalul AP 0.51–0.61: MLP Autoencoder este cel mai bun (AP 0.611), Isolation Forest cel mai slab (AP 0.511), dar diferența e suficient de mică încât nicio arhitectură nu arată un avantaj structural aici – anomaliile punctuale (vârfuri pe un singur pas temporal) nu necesită înțelegere temporală pentru a fi detectate.

4.2.2 A3 – Anomalii Contextuale/Sezoniere

Model	Paradigmă	ROC-AUC	Precizie Medie
Predictive LSTM	Predictiv	0.868	0.812
Transformer Bottleneck AE	Reconstrucție	0.762	0.663
Causal Transformer	Predictiv	0.738	0.623
MLP Autoencoder	Reconstrucție	0.765	0.507
LSTM Autoencoder	Reconstrucție	0.566	0.392
OC-SVM	Frontieră	0.584	0.376
Isolation Forest	Partiționare	0.511	0.331

Tabela 4.3: Yahoo S5 A3 (anomalii contextuale/sezoniere) – comparație de modele, clasificate după Precizia Medie.

Discuție. Rezultatele diverg puternic odată ce anomaliile devin contextuale – o valoare normală în termeni absoluți, dar greșită pentru faza sa în ciclul sezonier. Isolation Forest și OC-SVM se prăbușesc aproape de random (ROC-AUC 0.511 și, respectiv, 0.584). LSTM Autoencoder se prăbușește la fel de rău (ROC-AUC 0.566), în ciuda faptului că este un model secvențial – ceea ce arată că este vorba de o eșuare cauzată de blocajul de compresie (bottleneck), nu de o slăbiciune a arhitecturilor recurente în general. În aceeași ordine de idei, Transformer Bottleneck AE rămâne în urma celui mai bun model (AP 0.663), dar este de departe cea mai puternică arhitectură bazată pe *reconstrucție*, cu mult înaintea LSTM AE: auto-atenția globală asupra ferestrei recuperează probabil periodicitatea pe care blocajul LSTM AE o distruge, ceea ce probabil explică diferența. Predictive LSTM domină și este cel mai bun model pe orice benchmark Yahoo (AP 0.812, ROC-AUC 0.868): obiectivul său de predicție a pasului următor exploatează direct predictibilitatea unei serii sezoniere, astfel încât

o anomalie injectată produce un vârf clar în eroarea de predicție exact acolo unde violează traiectoria așteptată.

4.2.3 A4 – Anomalii de Tip Schimbare-de-Nivel

Model	Paradigmă	ROC-AUC	Precizie Medie
Predictive LSTM	Predictiv	0.745	0.505
Causal Transformer	Predictiv	0.695	0.436
MLP Autoencoder	Reconstrucție	0.719	0.402
Transformer Bottleneck AE	Reconstrucție	0.655	0.404
Isolation Forest	Partiționare	0.498	0.268
OC-SVM	Frontieră	0.519	0.265
LSTM Autoencoder	Reconstrucție	0.502	0.252

Tabela 4.4: Yahoo S5 A4 (anomalii de tip schimbare-de-nivel) – comparație de modele, clasificate după Precizia Medie.

Discuție. Seriile modelează acum un sistem care experimentează schimbări structurale bruște, iar fiecare model se degradează față de A3, indiferent de paradigmă. Un aspect important: ferestrele din toate cele ~ 100 de serii sunt grupate laolaltă fără identitatea seriei, iar fiecare serie se situează la propriul nivel de bază, deci valoarea absolută a unei ferestre, luată singură, nu poate distinge o fereastră autentică de după schimbare de o fereastră obișnuită dintr-o altă serie al cărei nivel de bază e similar – de unde rezultatele mai slabe ale modelelor bazate pe distribuția agregată. Modelele clasice pierd mult sub această confuzie – Isolation Forest ajunge la ROC-AUC 0.498. Autoencoderele de reconstrucție (MLP AE, Transformer Bottleneck AE) sunt afectate din același motiv și nu depășesc Predictive LSTM aici, în ciuda faptului că o fereastră în formă de treaptă ar fi intuitiv mai ușor de semnalat prin eroarea de reconstrucție. Predictive LSTM încă conduce (AP 0.505, ROC-AUC 0.745), deși marja sa se îngustează puternic față de A3.

4.2.4 Verificare de Referință Trivială (Doar Yahoo)

Urmând Kim et al. [6], care recomandă verificarea fiecărui model antrenat față de cel puțin o referință fără etichete (label-free), modelele Yahoo S5 au fost evaluate suplimentar față de o astfel de referință: norma L2 brută a ferestrei $\|w\|_2$, fără niciun model învățat. Toate modelele antrenate depășesc clar această referință pe A3, confirmând că avantajele observate mai sus sunt reale, nu un artefact al magnitudinii ferestrei. Scorurile referinței și marja exactă per model sunt prezentate în Anexa 5.

4.2.5 Discuție

Atât paradigma, cât și arhitectura contează. Rezultatele A3 formează o comparație de tip 2×2 între paradigmă (reconstrucție vs. predictiv) și arhitectură (LSTM vs. Transformer): reconstrucția obține 0.39 AP (LSTM AE) și 0.66 AP (Transformer Bottleneck), paradigma

predictivă obține 0.81 AP (Predictive LSTM) și 0.62 AP (Causal Transformer). Paradigma predictivă este necesară – ambele modele predictive depășesc ambele modele de reconstrucție – dar nu suficientă de una singură: diferența de 0.19 AP dintre Predictive LSTM și Causal Transformer, cu un obiectiv de antrenare identic, arată că arhitectura LSTM poartă un avantaj suplimentar pe date sezoniere. Pe A4, acest decalaj se reduce la 0.07 AP – o singură tranziție bruscă nu necesită acumulare de fază, deci biasul inductiv secvențial al LSTM-ului contează mai puțin.

Rezumat pe tip de anomalie. Anomaliile punctuale (A1) lasă toate modelele într-o bandă îngustă de AP 0.51–0.61 – nu e necesară nicio înțelegere temporală pentru a semnală un vârf izolat. Anomaliile sezoniere (A3) sunt locul unde avantajul paradigmei predictive e cel mai clar, și unde alegerea arhitecturii contează cel mai mult în cadrul acelei paradigme. Punctele de schimbare (A4) degradează fiecare model, dar Predictive LSTM încă conduce, iar Causal Transformer încă depășește ambele modele de reconstrucție. Modelele clasice (Isolation Forest, OC-SVM) și LSTM Autoencoder se prăbușesc pe orice tip de anomalie care necesită discriminare contextuală, nu absolută (A3, A4), rămânând competitive doar pe A1.

4.3 CIC-IDS2017 (Setul de Date C)

4.3.1 Comparație Generală

Fiecare valoare AP raportată mai jos este calculată pe întreaga mulțime de test flat/context agregată, în care PortScan, DoS Hulk și DDoS reprezintă împreună 95.1% din toate fluxurile de atac (compoziția completă pe clase: Tabelul 6.1, Anexa 6) – susținând afirmația, făcută repetat mai jos, că AP-ul agregat e practic un proxy pentru performanța pe aceste trei clase. Celelalte zece clase (toate sub 2% individual) au o influență limitată asupra clasamentului modelelor, de unde necesitatea unei detalieri pe atac (secțiunea 4.3.2). **DoS Hulk** (158,469 fluxuri) rămâne parte a clasei pozitive pentru fiecare metrică agregată, dar e exclus din detalieria *pe clasă* din secțiunea 4.3.2, întrucât Engelen, Rimmer, Joosen [3] arată că unealta folosită pentru a-l genera e configurată greșit (setează antetul `HTTP Connection` pe `Close` în loc de `Keep-Alive`), făcând atacul inefficient prin construcție.

Model	Set de caracteristici	Paradigmă	ROC-AUC	Precizie Medie	Acuratețe Echilibrată	F1 (atac)
MLP Autoencoder	Context	Reconstrucție	0.9860	0.9487	0.9548	0.9020
OC-SVM	Context	Frontieră	0.9745	0.9396	0.9569	0.9229
Isolation Forest	Context	Partiționare	0.9785	0.9328	0.9270	0.8367
LSTM Autoencoder	Temporal	Reconstrucție	0.8976	0.7955	0.7846	0.6249
MLP Autoencoder	Flat	Reconstrucție	0.9189	0.7921	0.8022	0.6305
Isolation Forest	Flat	Partiționare	0.9049	0.7612	0.8228	0.6639
OC-SVM	Temporal	Frontieră	0.8629	0.7816	0.7600	0.5888
MLP Autoencoder	Temporal	Reconstrucție	0.8831	0.7509	0.7734	0.6125
Isolation Forest	Temporal	Partiționare	0.8438	0.7604	0.6841	0.5195
OC-SVM	Flat	Frontieră	0.7336	0.6845	0.6121	0.4644
Bottleneck Transformer AE	Temporal	Reconstrucție	0.8077	0.6539	0.7059	0.5423
Causal Transformer	Temporal	Predictiv	0.7099	0.5449	0.5520	0.4320
Predictive LSTM	Temporal	Predictiv	0.6126	0.4472	0.5475	0.4295

Tabela 4.5: CIC-IDS2017 – toate combinațiile model/set-de-caracteristici, clasificate după Precizia Medie.

Caracteristicile de context al gazdei domină. Cele trei modele augmentate cu context ocupă primele trei poziții la fiecare metrică, cu AP între 0.933–0.949. Diferența dintre cel mai bun model cu context (AE+Context, AP 0.949) și cel mai bun model fără context (MLP AE Flat, AP 0.792) depășește întreaga plajă acoperită de celelalte zece modele. Fiecare dintre cele trei arhitecturi de bază se îmbunătățește substanțial la adăugarea caracteristicilor de context: IForest câștigă +0.172 AP (0.761 → 0.933), MLP AE câștigă +0.157 (0.792 → 0.949), iar OC-SVM câștigă cel mai mult, +0.255 (0.685 → 0.940) – semn că uneori caracteristicile mai bune contează mai mult decât alegerea paradigmei.

Adăugarea ferestrelor temporale, singură, nu ajută. Pentru IForest și MLP AE, gruparea fluxurilor în ferestre ordonate după IP-ul sursă (secțiunea 3.1) nu oferă o îmbunătățire semnificativă față de vederea flat, și uneori chiar dăunează – contrast direct cu câștigul din contextul gazdei: ambele modificări adaugă informație despre *ordinea/activitatea fluxurilor în timp*, dar doar statisticile de context agregate se traduc într-un semnal utilizabil pentru cele două clase de atac (DDoS, PortScan) care domină setul de test ca volum.

Reconstrucția depășește predicția. Printre modelele cu caracteristici temporale, arhitecturile bazate pe reconstrucție (LSTM AE, AE Temporal, OC-SVM Temporal) depășesc clar ambele modele bazate pe predicție (Predictive LSTM AP 0.447, Causal Transformer AP 0.545) – inversul constatării de la Yahoo A3 (capitolul 4). Sweep-urile de prag ale ambelor modele predictive ajung la un punct de operare extrem (FP > 1.1 mil. din 1.28 mil. de fluxuri benigne), consistent cu ROC-AUC-ul lor scăzut (0.61 și 0.71).

4.3.2 Detaliere pe Tip de Atac

Clasă de atac	<i>n</i>	IForest+Ctx	OC-SVM+Ctx	AE+Ctx
DDoS	95,000	0.982	0.998	0.994
PortScan	159,000	0.601	0.820	0.842
DoS GoldenEye	7,500	0.655	0.215	0.220
DoS slowloris	4,000	0.190	0.012	0.025
DoS Slowhttptest	1,700	0.112	0.007	0.013
Heartbleed	11	0.093	0.001	0.000
Infiltration	32	AP 0.03–0.14, fără un câștigător clar la niciun model		
Bot	738	<0.01 la toate modelele, inclusiv variantele cu context		
FTP-Patator	4,000	<0.01 la toate modelele, inclusiv variantele cu context		
SSH-Patator	3,000	<0.01 la toate modelele, inclusiv variantele cu context		
Subtipuri Web Attack	12–151	AP aproape zero peste tot (Brute Force, SQL Injection, XSS)		

Tabela 4.6: CIC-IDS2017 – Precizia Medie pe clasă de atac pentru cele trei modele augmentate cu context.

Detectate constant. Două clase de atac ies în evidență ca fiind detectabile fiabil pe aproape orice set de caracteristici:

- **DDoS** este cea mai ușoară clasă, cu o marjă mare: toate modelele cu context obțin AP > 0.98 (OC-SVM+Context atinge 0.998), iar chiar și modelele flat depășesc AP 0.77

– posibil pentru că rata mare de pachete mici, tipică unei inundații, produce o valoare aberantă extremă în ambele spații de caracteristici.

- **PortScan** este cea mai clară demonstrație a valorii caracteristicilor de context: fiecare model flat și temporal obține AP 0.07–0.28, în timp ce fiecare model cu context depășește 0.60, AE+Context atingând 0.842. Un singur flux de scanare scurt, către un port arbitrar, e nesemnificativ izolat, dar numărul de porturi de destinație unice contactate de IP-ul sursă în ultimele 60 de secunde ajută probabil în special la acest atac. IForest+Context e comparativ mai slab aici (0.601).

Răspuns mixt la caracteristici de context pentru DoS la nivel de aplicație. Nu fiecare clasă de atac răspunde la caracteristicile de context în același mod – cele trei clase DoS la nivel de aplicație împart familia de context în loc să o favorizeze uniform: **DoS GoldenEye** arată o separare clară: IForest+Context atinge AP 0.655, iar AE+Context și OC-SVM+Context se prăbușesc amândouă la ~ 0.22 . **DoS Slowhttptest** și **DoS slowloris** arată același tipar, mai atenuat: IForest+Context e singurul model cu context cu detecție semnificativă (AP 0.112 și 0.190), rămânând totuși mult sub AP-ul de 0.69/0.61 al IForest Temporal pe aceleași clase, probabil întrucât fereastra de context de 60s e o nepotrivire parțială pentru conexiunile lente, de minute întregi, ale acestor atacuri – ordonarea cronologică explicită per-gazdă a IForest Temporal rămâne cea mai bună detecție pentru această familie.

În mare parte nedetectabile, indiferent de setul de caracteristici. FTP-Patator, SSH-Patator, Bot și cele trei subtipuri Web Attack obțin toate AP < 0.01 la fiecare model, inclusiv cele cu context. Detectarea lor ar necesita cel mai probabil contoare de autentificare eșuată sau inspecția payload-ului, niciuna disponibilă în acest set. Heartbleed (11 eșantioane) și Infiltration (32) sunt prea mici pentru concluzii solide.

4.3.3 Discuție

Caracteristicile de context al gazdei sunt cea mai impactantă adăugare din acest studiu. Adăugarea a 9 statistici de activitate per-gazdă cu fereastră glisantă (secțiunea 3.1) ridică AP-ul de la 0.79 la 0.95 per ansamblu – un câștig mai mare decât alegerea paradigmei sau trecerea de la o vedere flat la una temporală – determinat în principal de rezolvarea punctului orb PortScan comun tuturor modelelor flat și temporale. Niciun model de context, luat singur, nu domină la fiecare tip de atac, deși AE+Context câștigă la AP-ul agregat, determinat de cele două clase cele mai mari.

Un ansamblu practic. Niciun model sau set de caracteristici, luat singur, nu acoperă fiecare clasă de atac din acest set de date. Combinarea AE+Context (puternic pe DDoS și PortScan, cele două clase care domină setul de test ca volum) cu IForest Temporal (puternic pe clasele DoS lente și, cu titlu provizoriu, pe Heartbleed) ar acoperi cea mai largă gamă de

clase de atac detectabile în cadrul acestui set de caracteristici. Această premisă, precum și un ansamblu cu scor combinat construit și evaluat pe baza ei, sunt testate direct în secțiunea 5.2: evidența celor două modele este confirmată ca fiind cu adevărat disjunctă, dar combinarea scorurilor lor se dovedește a schimba calitatea detecției PortScan pentru câștiguri pe clasele DoS lente, mai degrabă decât să le acopere pe amândouă gratuit.

Capitolul 5

Studii de Caz de Explicabilitate

Acest capitol testează afirmații specifice din Capitolul 4 prin diagnostice țintite, în loc să se bazeze doar pe metricile agregate.

5.1 Localizarea Erorii de Reconstrucție pe Yahoo A3

Discuția despre A3 din Capitolul 4 atribuie avantajul Transformer Bottleneck Autoencoder-ului față de LSTM Autoencoder (AP 0.663 vs. 0.392, ROC-AUC 0.813 vs. 0.566) unei diferențe dintre cele două bottleneck-uri: LSTM Autoencoder comprimă o fereastră întreagă de 64 de pași într-o singură stare ascunsă finală, ceea ce ar trebui să distrugă informația de fază sezonieră, în timp ce auto-atenția globală a Transformer-ului ar trebui să păstreze suficientă informație pozițională pentru a localiza eroarea precis, la pasul unde faza este încălcată. Se presupune deci că, pentru ferestrele A3 anormale, LSTM Autoencoder produce o eroare difuzată uniform pe toți pașii, în timp ce eroarea Transformer-ului prezintă un vârf ascuțit, localizat, la poziția unde tiparul sezonier e încălcat.

O verificare calitativă preliminară (hărți termice ale erorii per pas temporal – Anexa 7) e în linii mari consistentă cu ipoteza, dar insuficientă pentru a o confirma: fiecare panou e normalizat pe propria scală de culoare, iar valorile de eroare ale LSTM Autoencoder-ului sunt, ca magnitudine, aproximativ duble față de cele ale Transformer-ului pe tot parcursul. E necesară o măsură cantitativă, invariantă la scala absolută.

5.1.1 Verificare Cantitativă: Coeficientul Gini al Concentrării Erorilor

Pentru a separa concentrarea reală a erorii de un artefact al diferenței de magnitudine globală, cele 64 de erori pătratice per pas temporal ale fiecărei ferestre anormale au fost sumarizate printr-un coeficient Gini – o măsură standard de inegalitate (folosită inițial pentru distribuții de venit), invariantă la scala absolută a datelor. Pentru cele 64 de valori de eroare ale unei ferestre, sortate crescător, un Gini de 0 înseamnă eroare răspândită perfect uniform pe fiecare pas temporal, iar un Gini de 1 înseamnă concentrare totală la un singur pas.

Pentru n erori pătratice per pas temporal $x_1 \leq x_2 \leq \dots \leq x_n$ sortate crescător (aici $n = 64$), coeficientul Gini se calculează ca

$$G = \frac{2 \sum_{i=1}^n i x_i}{n \sum_{i=1}^n x_i} - \frac{n+1}{n}. \quad (5.1)$$

Calculat pe întreaga populație de 14,449 ferestre A3 anormale, cele două modele se separă clar:

	Gini Mediu	Gini Median
LSTM Autoencoder	0.566	0.606
Transformer Bottleneck Autoencoder	0.721	0.724

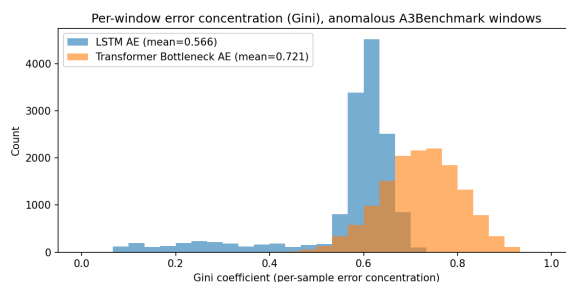


Figura 5.1: Distribuția coeficienților Gini per fereastră, pe toate cele 14,449 ferestre A3 anormale. Distribuția Transformer-ului este unimodală și deplasată spre concentrare ridicată (Gini > 0.6); distribuția LSTM Autoencoder-ului este bimodală, cu o majoritate grupată aproape de intervalul Transformer-ului, dar cu o populație secundară distinctă răspândită la valori Gini scăzute.

Aceasta confirmă ipoteza inițială în mare: eroarea Transformer-ului e mai concentrată decât cea a LSTM Autoencoder-ului pe întreaga populație anormală, nu doar în puținele exemple arătate întâmplător de o hartă termică. Dar distribuția din Figura 5.1 relevă un detaliu interesant – valorile Gini ale LSTM Autoencoder-ului sunt *bimodale*, nu deplasate uniform în jos. O majoritate a ferestrelor sale se grupează practic în același interval de concentrare ridicată ca al Transformer-ului (Gini ≈ 0.6 , capacitate de localizare comparabilă), în timp ce o populație secundară distinctă se situează la Gini scăzut (< 0.4), unde eroarea e cu adevărat difuzată aproape uniform pe toată fereastra. Aceasta indică faptul că mecanismul de distrugere a fazei nu e o proprietate uniformă a fiecărei reconstrucții LSTM Autoencoder, ci un fenomen care afectează un subset specific de ferestre – și această subpopulație, nu o degradare generală, e responsabilă pentru scăderea AP-ului și ROC-AUC-ului agregat al LSTM Autoencoder-ului sub cel al Transformer-ului, întrucât pe restul majoritar al ferestrelor cele două modele localizează eroarea la fel de bine.

5.2 Regimuri de Caracteristici Disjuncte: AE+Context vs. Temporal IForest

Discuția despre CIC-IDS2017 din Capitolul 4 propune combinarea AE+Context (puternic pe DDoS și PortScan) cu Temporal IForest (puternic pe clasele DoS lente și, cu rezerve, pe Heartbleed) într-un ansamblu, întrucât cele două ar trebui să opereze pe mulțimi de caracteristici disjuncte – caracteristici de context al gazdei versus o fereastră temporală de 16 fluxuri.

O scurtă introducere la SHAP. SHAP (*SHapley Additive exPlanations*) atribuie fiecărui eșantion câte o valoare per caracteristică de intrare, reprezentând cât de mult a influențat acea caracteristică ieșirea modelului. Semnul arată direcția împingerii (spre normal sau spre anormal, în funcție de convenția ieșirii explicate – precizată separat pentru fiecare model mai jos, întrucât cele două nu coincid), iar magnitudinea arată contribuția relativă la celelalte caracteristici ale aceluiași eșantion.

O *diagramă beeswarm* (Figurile 9.2 și 5.3) arată atribuirea SHAP pe un eșantion de puncte de test, câte un rând per caracteristică, ordonate descrescător după importanță; pe fiecare rând, fiecare punct e un eșantion, poziționat orizontal după propria valoare SHAP și colorat după propria valoare a caracteristicii (albastru pentru valoare mică, roșu pentru valoare mare). Pentru definiția matematică și originea din teoria jocurilor cooperative: [7]; pentru o descriere mai accesibilă și detalii despre construcția diagramelor: documentația SHAP [8].

Se presupune deci că (1) dacă Temporal IForest exploatează cu adevărat structura dintre pașii temporali, scorul său per fereastră ar trebui să poarte informație despre eticheta *ultimului* flux – cel combinat cu scorul lui AE+Context – nu doar despre dacă *vreun* flux din fereastra de 16 e un atac, deci atribuirea sa SHAP nu ar trebui să se prăbușească pe acel flux final; și (2) avantajul lui AE+Context pe propriile sale clase puternice provine din cele 9 caracteristici de context al gazdei, care ar trebui deci să apară printre caracteristicile cu cea mai mare medie |SHAP|.

O validare suplimentară – verificarea faptului că scorul lui Temporal IForest rămâne informativ și sub o etichetare mai strictă, per ultimul flux, nu doar sub eticheta permisivă „oricare-din-16” – este detaliată în Anexa 8.

5.2.1 Temporal IForest: Atribuire SHAP pe Toată Fereastra

Temporal IForest e antrenat pe ferestre aplatizate de 16 fluxuri, deci valorile SHAP ale lui TreeExplainer [9] pe intrarea aplatizată se descompun curat înapoi într-un cub (fereastră, pas temporal, caracteristică). Pentru fiecare din cele patru clase, SHAP a fost calculat pe 200 de ferestre; magnitudinea atribuirii a fost sumată pe caracteristici per pas temporal, iar caracteristicile cu cea mai mare contribuție au fost reprezentate pe toți cei 16 pași pentru a verifica dacă poartă o atribuire cu semn consistent pe toată fereastra, nu doar aproape de final.

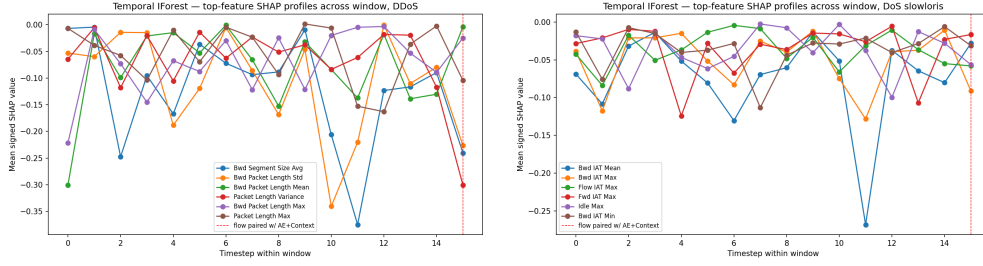


Figura 5.2: Temporal IForest – valoarea medie SHAP per pas temporal pentru cele șase caracteristici cu cea mai mare atribuire, ferestre DDoS (stânga) și ferestre DoS slowloris (dreapta). În ambele cazuri atribuirea este negativă (anormală), recurentă cu semn consistent pe majoritatea celor 16 pași temporali, și nu se concentrează pe fluxul final (linia roșie punctată) – fluxul combinat cu scorul lui AE+Context în ansamblul propus.

Doi diagnostici suplimentari (Anexa 9) confirmă că ultimul pas temporal nu domină atribuirea (6–8% din masa totală, aproape de referința uniformă de 6.25% pentru o fereastră de 16 pași). Caracteristicile principale se potrivesc cu mecanismul cunoscut al fiecărui atac: atribuirea DDoS/GoldenEye se concentrează pe lungimea pachetelor pe direcția inversă (**Bwd Packet Length Std/Mean/Max**), consistent cu un flux volumetric; atribuirea Slowhttpstest/slowloris se concentrează pe timpul între sosiri (**Fwd/Bwd IAT Std/Max**), consistent cu comportamentul lent, de menținere a conexiunii – confirmând ipoteza.

5.2.2 AE+Context: SHAP Dominat de Caracteristici de Context al Gazdei

SHAP a fost calculat cu `GradientExplainer` pe eroarea de reconstrucție a lui AE+Context, pe câte 200 de eșantioane din PortScan și DDoS, iar cele 15 caracteristici cu cea mai mare medie $|\text{SHAP}|$ au fost clasate pe clasă.

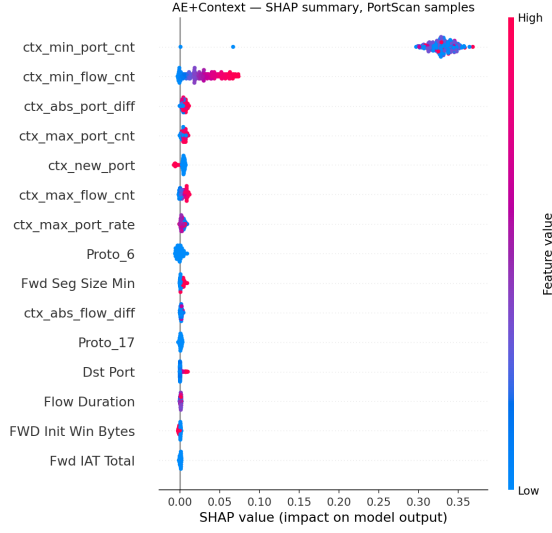


Figura 5.3: AE+Context – diagramă sumară SHAP, eşantioane PortScan. `ctx_min_port_cnt` domină printr-un cluster strâns, uniform pozitiv, bine separat de orice altă caracteristică; rândurile de top rămase sunt aproape în întregime caracteristici de context al gazdei.

Caracteristici de context în top 15		Caracteristica principală (medie $ \text{SHAP} $)
PortScan	8/15	<code>ctx_min_port_cnt</code> (0.324)
DDoS	7/15	<code>ctx_min_port_cnt</code> (0.234)

Pentru ambele clase, `ctx_min_port_cnt` și `ctx_min_flow_cnt` domină cu aproximativ un ordin de mărime peste orice caracteristică de flux obișnuită. Șase din cele nouă caracteristici de context apar în listele de top 15 ale ambelor clase, doar `ctx_min_port_rate` lipsind din amândouă – confirmând premisa: avantajul lui AE+Context pe aceste clase e determinat aproape în întregime de dovezi de context al gazdei indisponibile lui Temporal IForest.

5.2.3 Construirea Scorului Combinat

Realizarea ansamblului schițat în Capitolul 4 necesită combinarea, pentru fiecare flux f , a scorului propriu al lui AE+Context cu scorul lui Temporal IForest pentru fereastra de 16 fluxuri care se *încheie* la f . Cele două scoruri brute nu pot fi combinate direct (eroare de reconstrucție fără scală fixă vs. lungime medie de cale negată), deci e necesară o normalizare – calculată doar pe scorurile de antrenare *benigne* ale fiecărui model, niciodată pe date de test, pentru a evita scurgerea informației de anomalie în combinare.

Au fost testate două reguli: **votarea ponderată (soft voting)**, $s_{\text{soft}}(f) = w \cdot \hat{s}_{AE}(f) + (1 - w) \cdot \hat{s}_{IF}(f)$, variind $w \in \{0.3, \dots, 0.7\}$; și **regula maximului**, $s_{\text{max}}(f) = \max(\hat{s}_{AE}(f), \hat{s}_{IF}(f))$ – un combinator de tip OR, reflectând ideea că încrederea oricărui model, singură, ar trebui să fie suficientă.

Cele două scoruri au fost normalizate pe rang – mapând fiecare scor de test la percentila sa în distribuția scorurilor proprii de antrenare benignă – în locul standardizării z , care s-a dovedit o fundătură (normalizarea pe rang mărginește fiecare scor la $[0, 1]$, indiferent de valoarea brută). Detalii: Anexa 10.

5.2.4 Acoperire per Clasă vs. AP Agregat

	DDoS	PortScan	GoldenEye	Slowhttptest	slowloris
AE+Context singur	0.882	0.924	0.292	0.013	0.025
Temporal IForest singur	0.956	0.123	0.611	0.578	0.560
Regula maximului (rang)	0.882	0.923	0.292	0.060	0.083
Votare ponderată $w = 0.4$ (rang)	0.977	0.228	0.801	0.205	0.319
Votare ponderată $w = 0.7$ (rang)	0.977	0.358	0.821	0.140	0.245

AP-ul agregat *favorizează* de fapt variantele care se schimbă cel mai puțin: regula maximului (0.943) și AE+Context singur (0.941) depășesc votarea ponderată $w = 0.4$ (0.841), pur și simplu pentru că DDoS și PortScan totalizează peste 250,000 din cele $\sim 434,000$ de fluxuri pozitive, față de doar 13,000 pentru GoldenEye, Slowhttptest și slowloris împreună.

Nicio regulă nu e un câștig gratuit, dar votarea ponderată pare alegerea mai pertinentă pentru un ansamblu motivat de acoperire: clasa sa cea mai *slabă* (0.205, Slowhttptest, la $w = 0.4$) depășește propriile clase cele mai slabe ale ambelor modele singure – 0.013 (AE+Context) și 0.123 (Temporal IForest) – ceva ce niciun model singur sau regula maximului nu realizează. E o realizare autentică, deși parțială, a ideii de ansamblu din Capitolul 4: votarea ponderată pe rang lărgeste pragul minim de acoperire, cu costul detecției PortScan.

Capitolul 6

Concluzii

Întrebarea formulată în Capitolul 1 a fost în ce măsură modul în care un model reprezintă structura latentă a datelor „normale” determină tipurile de anomalii pe care le poate detecta. Rezultatele din Capitolul 4, confirmate și nuanțate în Capitolul 5, dau un răspuns afirmativ: aceeași paradigmă produce rezultate diferite pe seturi de date diferite, nu pentru că modelul ar fi mai bun sau mai slab în sine, ci pentru că presupunerea lui despre normalitate se potrivește sau nu structurii reale a anomaliei căutate.

Cele două ipoteze din §2.1.1 se confirmă, dar nu necondiționat. Prima – structura temporală avantajează datele secvențiale – se confirmă pe Yahoo A3/A4 (paradigma predictivă domină), dar se inversează pe Setul A și e depășită de caracteristicile de context la CIC-IDS2017. A doua – paradigme complementare, combinate, îmbunătățesc performanța – e confirmată de studiul din §5.2: AE+Context și Temporal IForest exploatează evidențe disjuncte, iar votarea ponderată extinde acoperirea pe clase altfel nedetectate, cu costul unui AP agregat mai mic – nuanțând, nu infirmând, ipoteza.

Limitările lucrării deschid două direcții de continuare: testarea sistematică a celorlalte presupuneri din discuțiile Capitolului 4 – de exemplu de ce ferestrele temporale dăunează performanței la Setul de Date A și C, sau de ce clase precum FTP-Patator, SSH-Patator, Bot rămân nedetectabile indiferent de setul de caracteristici; și extinderea studiului de ansamblu din §5.2.3 la Setul de Date A și B, cu reguli de combinare mai sofisticate decât votarea ponderată și regula maximului.

Anexa 1

Detalii ale Mecanismelor de Model

Această anexă completează §2.2 cu detaliile mecanismelor fiecărui model, omise din textul principal pentru concizie.

1.1 One-Class SVM

Cei trei hiperparametri fixați identic pe toate seturile de date, urmând implementarea `scikit-learn` [10], determină împreună forma graniței învățate:

- `kernel='rbf'` fixează spațiul de caracteristici: fiecare punct de antrenare generează un **clopot gaussian** neted în jurul său, iar granița finală este conturul închis format prin suprapunerea acestor clopote în jurul zonelor dense de date.
- `gamma='scale'` stabilește cât de largă este fiecare clopot gaussian, deci cât de strâns se poate curba granița în jurul structurii locale a datelor – o valoare mai mare permite o graniță mai strânsă, dar cu risc mai mare ca modelul să învețe zgomotul.
- `nu=0.01` controlează cât de strâns este forțată granița să încadreze datele de antrenare, limitând la $\sim 1\%$ fracția de puncte benigne pe care modelul are voie să le lase în afara ei.

1.2 Isolation Forest

Numărul de arbori este dat de parametrul `n_estimators` (implementarea `scikit-learn` [11]). Deoarece diviziunile sunt aleatoare, punctele ușor de separat de restul datelor – anomaliile, care tind să se afle în regiuni rare, de densitate scăzută – sunt izolate în mai puține diviziuni. Modelul returnează lungimea medie negată a căii, care este negată încă o dată aici, astfel încât un scor mai mare să însemne o cale mai scurtă (mai anormal), păstrând convenția consistentă cu toate celelalte modele. Parametrul `contamination` ar seta în mod normal pragul intern de decizie pe baza unei fracții presupuse de anomalii în datele de antrenare, dar cum datele de antrenare aici sunt exclusiv benigne, iar pragul de operare este ales extern, `contamination` nu are niciun efect asupra rezultatelor raportate – contează doar scorul de anomalie continuu.

1.3 Autoencodere (MLP, LSTM) și LSTM Predictiv

Pentru Autoencoder Feedforward, dropout-ul este aplicat între straturile ascunse ale perechii encoder/decoder.

Pentru Autoencoder LSTM, starea de celulă (nu doar starea ascunsă) este de asemenea păstrată ca parte a vectorului sumar transmis decoderului; starea ascunsă finală a encoderului este repetată identic la fiecare pas de timp ca intrare pentru decoder, ale cărui ieșiri per pas de timp sunt proiectate înapoi la dimensionalitatea originală printr-un strat liniar comun.

Pentru LSTM Predictiv, modelul nu vede niciodată valoarea pe care este pus să o prezică; fiecare stare ascunsă de-a lungul parcursului este folosită pentru a produce o predicție, nu doar cea finală, spre deosebire de Autoencoder LSTM, unde doar starea finală contează.

1.4 Autoencodere Bazate pe Transformer

- **Bottleneck Transformer Autoencoder.** Fiecare caracteristică/pas de timp de intrare este proiectat(ă) într-un token de dimensiune D_{MODEL} și trecut(ă) prin encoderul transformer cu auto-atenție completă (non-cauzală), astfel încât fiecare poziție poate atenționa liber orice altă poziție. Reprezentările rezultate per poziție sunt apoi mediate global de-a lungul secvenței într-un singur vector de dimensiune D_{MODEL} , comprimat printr-un strat liniar la un $LATENT_DIM$ strict mai mic, apoi extins înapoi printr-un al doilea strat liniar la forma completă a secvenței aplatizate. Acest pas de comprimare-extindere este bottleneck-ul explicit de informație care împiedică mecanismul de atenție să copieze pur și simplu intrarea în ieșire. Scor de anomalie: MSE per eșantion între intrare și reconstrucție, pe toate pozițiile deodată – întreaga secvență reconstruită este produsă printr-o singură trecere prin rețea, nu pas cu pas ca la un decoder recurent.
- **Causal Transformer.** Este folosită aceeași structură de encoder, dar cu o mască de atenție cauzală (triunghiulară superior) aplicată, astfel încât poziția t poate atenționa doar pozițiile $0..t$; modelul este antrenat cu același obiectiv de predicție a pasului următor ca LSTM Predictiv, fără bottleneck de comprimare. Scor de anomalie: MSE per eșantion între valorile prezise și cele reale ale pasului următor.

Anexa 2

Detalii Suplimentare de Preprocesare

Această anexă completează Secțiunea 3.1 cu pașii exacti ai fiecărui pipeline de preprocesare.

2.1 Preprocesare Tabelară (Credit Card)

1. **Time** este eliminat (nu reprezintă informație semnificativă, întrucât este doar timpul de la prima tranzacție din setul de date).

2. Rândurile sunt împărțite pe clase: toate rândurile benigne (**Class=0**) sunt reunite, apoi împărțite 80/20 în antrenare și test-benign. Setul de antrenare este partiția de 80%, conținând doar rânduri benigne (~227,451 rânduri).
3. Setul de testare este format din restul de 20% rânduri benigne, plus *toate* cele 492 de rânduri de fraudă.
4. Scalarea este asimetrică, pe grupuri de caracteristici: **V1--V28** trec **nescalate** implicit (**V_SCALING** = 'none' – întrucât acestea sunt deja caracteristici PCA), cu 'standard' încercat de asemenea în unele rulări, fără o diferență reală între modele. **Amount** este scalat independent cu un **RobustScaler** (bazat pe mediană/IQR), ales pentru robustețea sa la coada dreaptă accentuată, urmând același raționament ca notebook-ul exploratoriu citat în §2.

2.2 Crearea de Ferestre pentru Serii de Timp (Yahoo S5)

1. Fiecare serie este împărțită 70/30 de-a lungul axei temporale (**TRAIN_FRAC=0.70**) – o împărțire temporală și nu o amestecare aleatoare. Nu vrem ca vreo informație din viitor să se scurgă în antrenare.
2. Un **MinMaxScaler** este potrivit *per serie*, folosind doar punctele neanormale din partiția de antrenare a acelei serii, apoi aplicat atât partiției de antrenare, cât și celei de testare ale aceleiași serii.
3. Se aplică o fereastră glisantă de lungime **WINDOW=64**: pas 4 pentru ferestrele de antrenare (subeșantionate, păstrându-se pentru antrenare doar ferestrele complet normale) și pas 1 pentru ferestrele de testare (dens – este evaluată fiecare poziție posibilă a ferestrei).
4. Eticheta unei ferestre urmează regula „oricare”: pozitivă dacă cel puțin unul dintre cei 64 de pași de timp este anormal. Aceasta este totuși o sursă de „estompare” a etichetelor – de exemplu, pe A1, o rată de anomalii la nivel de punct de 1.76% devine o rată la nivel de fereastră de ~17–18%, întrucât un singur punct anormal contaminează până la 64 de ferestre suprapuse. O fereastră mai mică ar reduce această estompare, lucru care ar putea merita încercat în viitor.
5. Ferestrele din toate seriile unui benchmark (reamintesc că fiecare benchmark este alcătuit din mai multe serii univariate) sunt reunite în array-uri unice.

Comparație cu DeepAnt. DeepAnt [4] a fost folosit ca referință pentru evaluarea unui predictor neural al pasului următor pe Yahoo S5, deși mai multe alegeri metodologice diferă de cele ale acestui proiect:

- **Predicție many-to-one, nu many-to-many.** Predictorul propus de DeepAnt este o rețea convoluțională; varianta LSTM raportată de aceștia ca referință de comparație (două straturi suprapuse, de câte 32 de unități) este mai îngustă și mai puțin adâncă decât LSTM-ul de 64 de unități folosit aici pentru LSTM Predictiv. Mai important, ambele prezic doar un singur pas de timp următor dintr-o fereastră (many-to-one), în loc de fiecare pas $t+1$ din $0..t$ simultan, așa cum fac modelele predictive ale acestui proiect (§2).
- **Antrenare per serie vs. antrenare reunită.** DeepAnt antrenează și evaluează un model separat pentru fiecare serie de timp individuală (o împărțire 40/60 antrenare/test per serie, scorul F calculat per serie și apoi mediat per sub-benchmark), ceea ce permite fiecărui model să se specializeze pe o singură frecvență. Acest proiect reunește în schimb ferestrele din toate cele ~ 100 de serii ale unui benchmark într-un singur model comun, antrenat pe o împărțire 70/30 – în principal din motive de simplitate a implementării.

2.3 Preprocesarea Fluxurilor de Rețea (CIC-IDS2017)

Există trei implementări pentru acest set de date, toate având la bază aceiași pași de curățare (eliminarea Flow ID/Src IP/Dst IP/Timestamp/Src Port; codificare one-hot pentru Protocol; înlocuirea Inf cu NaN; imputare cu mediana, calculată pe setul de antrenare; eliminarea coloanelor cu varianță zero, măsurată pe setul de antrenare; MinMaxScaler potrivit doar pe setul de antrenare), dar diferind prin unitatea de observație:

Plat Un rând = un flux. Setul de antrenare este format din toate fluxurile benigne din ziua de luni (371,749 rânduri); setul de testare conține toate fluxurile benigne de marți–vineri, plus toate fluxurile de atac reale (fluxurile -- **Attempted**, adică atacuri fără încărcătură observată, sunt excluse complet). Filtrarea coloanelor cu varianță zero pe datele de luni elimină cele trei coloane de flag URG, rămânând 77 din cele 80 de caracteristici originale.

Temporal Un rând = o fereastră de WINDOW=16 fluxuri consecutive de la aceeași adresă IP sursă, sortate după timestamp. Această alegere de grupare – după IP sursă, nu după sesiunea individuală de flux (identificată printr-un 5-tuplu: IP sursă, IP destinație, port sursă, port destinație și protocol – cei cinci identificatori pe baza cărora CICFlowMeter definește o conexiune individuală) – corespunde cheii de grupare folosite și pentru calculul caracteristicilor de context de gazdă de mai jos. Ambele pipeline-uri tratează „o entitate” drept fluxul de activitate ordonat al unei gazde sursă, nu o conexiune 5-tuplu, întrucât comportamentul la nivel de gazdă (multe conexiuni scurte, în rafală) caracterizează atacuri precum PortScan sau DoS, mai degrabă decât structura internă a unei singure conexiuni. Împărțirea este pe zile (luni antrenare, marți–vineri testare), din același motiv ca la pipeline-ul plat.

Context de gazdă Aceeași curățare plată, per flux, ca mai sus, dar fiecare flux este suplimentat cu 9 caracteristici construite, calculate dintr-o fereastră temporală glisantă de 60 de secunde, per IP gazdă: numărul de fluxuri, numărul de porturi unice, rata de diversitate a porturilor, un indicator de port nou, precum și asimetria sursă/destinație a fiecăreia. Acestea sunt calculate cauzal (doar din activitatea *anterioară* fluxului curent) și nu necesită etichete de atac, urmând adaptarea nesupervizată a lui Raskovalov, Gabdullin, Androsoy [5]. Numărul final de caracteristici: $77 + 9 = 86$. Această variantă a fost introdusă special pentru a rezolva o slăbiciune comună pipeline-urilor plat și temporal, pe clasa de atac PortScan (vezi §4).

Anexa 3

Alegeri legate de Hiperparametri și Antrenare

Două decizii principale se aplică uniform pentru toate modelele și seturile de date:

- **Optimizator.** Toate rețelele neuronale sunt antrenate cu Adam, la o rată de învățare fixă de 10^{-3} , uniform pentru fiecare model și set de date.
- **Fără căutare de hiperparametri.** Dacă un model a avut performanțe neașteptat de slabe, rata sa de învățare a fost verificată suplimentar cu $\{10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$ înainte de a trage vreo concluzie din acel rezultat.
- **Partiție de validare doar pentru monitorizare.** Fiecare rețea neuronală reține aleator 10% din datele de antrenare exclusiv pentru a trasa o curbă de pierdere (*loss*) de antrenare/validare per rulare, ca verificare că antrenarea decurge corect. Această partiție nu este niciodată folosită pentru oprire timpurie (*early stopping*) sau selecția modelului: antrenarea rulează întotdeauna pentru numărul fix de epoci de mai jos, iar rezultatele raportate pe setul de testare folosesc ponderile modelului din epoca finală.

3.1 Isolation Forest

	CIC flat	CIC context	Yahoo	Credit card
N_ESTIMATORS	150	150	150	150
MAX_SAMPLES	300,000	300,000	$\min(n, 50,000)$	200,000
Dimensiunea setului de antrenare	~371k	~371k	câteva mii	~227k
% din antrenare folosit	~81%	~81%	~100%	~88%
RANDOM_SEED	123	123	123	123

MAX_SAMPLES este singura diferență semnificativă și este setat cât mai ridicat posibil, practic, pentru fiecare set de date (acuratețea crește odată cu mai multe eșantioane per arbore, cu randamente descrescând; problema de „swamping” al arborilor nu apare aici, întrucât datele de antrenare sunt întotdeauna exclusiv benigne). Plafonul de 50.000 pentru Yahoo nu este atins niciodată în practică și este păstrat doar ca limită de siguranță.

3.2 One-Class SVM

	CIC flat	CIC context	Yahoo	Credit card
NU	0.01	0.01	0.01	0.01
KERNEL	rbf	rbf	rbf	rbf
GAMMA	scale	scale	scale	scale
SUBSAMPLE	50,000	50,000	niciunul (complet)	niciunul (complet)
% din antrenare folosit	~13.5%	~13.5%	~100%	~100%

Toți parametrii kernel/nu sunt uniformi pe seturile de date. Subeșantionarea este aplicată doar acolo unde setul de antrenare este suficient de mare încât un OC-SVM pe toate datele ar deveni mult prea lent; credit card (~227k rânduri) a fost antrenat pe tot setul, fără subeșantionare.

3.3 Autoencoder Feedforward (MLP)

	CIC flat	CIC context	Yahoo	Credit card
Dimensiune intrare	77	86	64 (fereastră aplatizată)	~30 (PCA)
Dimensiuni ascunse	[64, 32, 16]	[64, 32, 16]	[32, 16]	[16, 8]
Raport bottleneck	0.21	0.19	0.25	~0.27
Activare finală	Sigmoid	Sigmoid	Linear	Linear

Lățimile straturilor sunt scalate în funcție de dimensiunea intrării, astfel încât raportul de compresie al bottleneck-ului să rămână aproximativ constant (~0.20–0.27) pe toate seturile de date. Activarea finală diferă întrucât CIC folosește `MinMaxScaler` (mărginit la $[0, 1]$, potrivit pentru o ieșire sigmoid), în timp ce Yahoo și credit card folosesc scalare nemărginite (ieșire liniară).

3.4 LSTM Predictiv

	CIC	Yahoo	Credit card
INPUT_SIZE	77	1	1
HIDDEN_DIM	128	64	64
NUM_LAYERS	1	1	1
EPOCHS	20	20	50
BATCH_SIZE	256	256	512

`HIDDEN_DIM` este scalat în funcție de dimensiunea intrării pentru CIC (o extindere de $\sim 1.66\times$ față de cele 77 de caracteristici de intrare); ambele seturi de date univariate folosesc 64, o capacitate suficientă pentru un semnal scalar. Credit card folosește mai multe epoci (50) întrucât curba sa de pierdere continuă să scadă și după epoca 30 – probabil o consecință directă a tratării celor 29 de caracteristici PCA netemporale ca pseudo-secvență, ceea ce face optimizarea mai zgomotoasă decât pe date cu adevărat temporale. CIC și Yahoo converg până la epoca ~5, deci 20 de epoci este deja o alegere conservatoare pentru acestea.

3.5 Autoencoder LSTM

	CIC	Yahoo	Credit card
INPUT_SIZE	77	1	1
HIDDEN_DIM	128	64	64
NUM_LAYERS	1	1	1
EPOCHS	50	30	50
BATCH_SIZE	256	256	512

Atât CIC, cât și credit card au necesitat 50 de epoci pentru a atinge un platou; Yahoo converge până la epoca ~ 5 , deci 30 este deja generos.

3.6 Bottleneck Transformer Autoencoder

	CIC	Yahoo	Credit card
D_MODEL	64	64	64
NHEAD	4	4	4
NUM_LAYERS	2	2	2
DIM_FF	256	256	256
LATENT_DIM	32	32	32
EPOCHS	50	20	20
BATCH_SIZE	256	256	512

LATENT_DIM=32 oferă o compresie uniformă de 50% față de D_MODEL=64, pe toate cele trei seturi de date.

3.7 Causal Transformer

	CIC	Yahoo
D_MODEL	64	64
NHEAD	4	4
NUM_LAYERS	2	2
DIM_FF	256	256
DROPOUT	0.1	0.1
EPOCHS	30	30
BATCH_SIZE	256	256

Anexa 4

Protocolul de Evaluare: Metrici și Rezultate Complete

4.1 Metrici Raportate

Fiecare rulare produce un raport text care conține toate metricile din Tabelul 4.1. Capitolul de Rezultate raportează doar un subset dintre acestea (de obicei ROC-AUC, Precizia Medie, și, unde este relevant, Acuratețea Echilibrată și F1), ales pentru a nu aglomera tabelele de comparație; setul complet, per model și per set de date, este prezentat în §4.2–§4.4.

Metrică	Note
Precizie / Recall / F1 (per clasă)	Prin <code>sklearn.classification_report</code> , raportate atât pentru clasa normală, cât și pentru clasa anomalie, plus acuratețea și mediile macro/ponderate.
ROC-AUC	Raportat pentru completitudine / comparabilitate cu literatura anterioară; nu este metrica principală de clasificare în condiții de dezechilibru de clase.
Precizie Medie (PR-AUC)	Independentă de prag; alegerea standard în condiții de dezechilibru sever de clase, întrucât ROC-AUC poate părea înșelător de bun atunci când exemplele negative depășesc masiv numărul celor pozitive.
Acuratețe Echilibrată	$\frac{1}{2}(\text{sensibilitate} + \text{specificitate})$ la pragul de operare selectat.
F1 (clasa pozitivă/atac)	La pragul de operare selectat.
Numărători din matricea de confuzie	TP/TN/FP/FN brute, la pragul selectat.

Tabela 4.1: Metrici raportate de fiecare rulare de evaluare.

Notă privind consistența numerelor. Tabelele 4.2–4.4 de mai jos completează Capitolul 4 cu detaliul complet (precizie/recall pe clasa pozitivă, matrice de confuzie) pentru fiecare model. Coloanele ROC-AUC / Precizie Medie / Acuratețe Echilibrată / F1 reproduc exact valorile deja publicate în Capitolul 4, pentru consistență. Pentru un număr de modele – în special pe CIC-IDS2017, unde reevaluări ulterioare au schimbat scorurile – nu toate metricile au mai fost calculate. În aceste cazuri, rândul din tabel afișează doar metricile agregate deja publicate.

4.2 Rezultate Complete – Credit Card Fraud Detection

Model	Precizie (fraudă)	Recall (fraudă)	F1 (fraudă)	ROC-AUC	Precizie Medie	Acuratețe Echilibrată	TP	TN	FP	FN
MLP Autoencoder	0.6205	0.8008	0.6992	0.9649	0.7105	0.8983	394	56622	241	98
Isolation Forest	0.5000	0.7724	0.6070	0.9587	0.6725	0.8828	380	56483	380	112
LSTM Autoencoder	0.6904	0.7886	0.7362	0.9476	0.6223	0.8928	388	56689	174	104
OC-SVM	0.5501	0.7805	0.6454	0.9354	0.5959	0.8875	384	56549	314	108
Transformer Bottleneck AE	0.5073	0.7764	0.6137	0.9583	0.5947	0.8849	382	56492	371	110
Predictive LSTM	0.3992	0.7642	0.5244	0.9581	0.5190	0.8771	376	56297	566	116

Tabela 4.2: Credit Card Fraud Detection – toate metricile raportate, pentru toate cele șase modele evaluate.

4.3 Rezultate Complete – Yahoo Webscope S5

Model	Precizie (poz.)	Recall (poz.)	F1 (poz.)	ROC-AUC	Precizie Medie	Acuratețe Echilibrată	TP	TN	FP	FN
A1 – anomalii punctuale										
MLP Autoencoder	–	–	–	0.792	0.611	–	–	–	–	–
LSTM Autoencoder	0.2944	0.7904	0.4290	0.794	0.600	0.6916	852	2974	2042	226
OC-SVM	0.5183	0.5909	0.5522	0.776	0.595	0.7364	637	4424	592	441
LSTM Predictiv	0.2988	0.8016	0.4354	0.789	0.588	0.6988	3442	11912	8076	852
Isolation Forest	0.2439	0.8646	0.3805	0.765	0.511	0.6443	932	2127	2889	146
A3 – anomalii contextuale/sezoniere										
LSTM Predictiv	–	–	–	0.868	0.812	–	–	–	–	–
Transformer Bottleneck AE	–	–	–	0.762	0.663	–	–	–	–	–
Causal Transformer	–	–	–	0.738	0.623	–	–	–	–	–
MLP Autoencoder	0.4247	0.9540	0.5878	0.765	0.507	0.6622	13785	10981	18670	664
LSTM Autoencoder	0.3278	1.0000	0.4937	0.566	0.392	0.5003	14449	20	29631	0
OC-SVM	0.3285	0.9996	0.4945	0.584	0.376	0.5020	14443	128	29523	6
Isolation Forest	0.3301	0.9960	0.4959	0.511	0.331	0.5055	14391	447	29204	58
A4 – schimbări de nivel										
LSTM Predictiv	0.3975	0.7951	0.5300	0.745	0.505	0.6864	9098	18868	13789	2345
Causal Transformer	0.3092	0.9098	0.4616	0.695	0.436	0.5988	10411	9401	23256	1032
MLP Autoencoder	–	–	–	0.719	0.402	–	–	–	–	–
Transformer Bottleneck AE	0.2625	0.9959	0.4155	0.655	0.404	0.5077	11396	636	32021	47
Isolation Forest	0.2595	1.0000	0.4121	0.498	0.268	0.5001	11443	6	32651	0
OC-SVM	0.2603	1.0000	0.4131	0.519	0.265	0.5021	11443	139	32518	0
LSTM Autoencoder	0.2644	0.9979	0.4181	0.502	0.252	0.5127	11419	895	31762	24

Tabela 4.3: Yahoo Webscope S5 – toate metricile raportate, pe benchmark (A1, A3, A4), ordonate ca în tabelele din Capitolul 4.

4.4 Rezultate Complete – CIC-IDS2017

Compoziția pe clase de atac a setului de test temporal (analoagă Tabelului 6.1 din Capitolul 4 pentru setul flat/context) este detaliată separat, în Capitolul 6.

Model	Set de caracteristici	Precizie (atac)	Recall (atac)	F1 (atac)	ROC-AUC	Precizie Medie	Acuratețe Echilibrată	TP	TN	FP	FN
MLP Autoencoder	Context	–	–	–	0.9860	0.9487	0.9548	–	–	–	–
OC-SVM	Context	0.8965	0.9509	0.9229	0.9745	0.9396	0.9569	412650	1238310	47634	21327
Isolation Forest	Context	–	–	–	0.9785	0.9328	0.9270	–	–	–	–
LSTM Autoencoder	Temporal	0.4752	0.9126	0.6249	0.8976	0.7955	0.7846	398793	842158	440453	38212
MLP Autoencoder	Flat	–	–	–	0.9189	0.7921	0.8022	–	–	–	–
Isolation Forest	Flat	–	–	–	0.9049	0.7612	0.8228	–	–	–	–
OC-SVM	Temporal	0.4214	0.9770	0.5888	0.8629	0.7816	0.7600	426967	696384	586227	10038
MLP Autoencoder	Temporal	–	–	–	0.8831	0.7509	0.7734	–	–	–	–
Isolation Forest	Temporal	0.3577	0.9487	0.5195	0.8438	0.7604	0.6841	414606	538115	744496	22399
OC-SVM	Flat	0.3045	0.9778	0.4644	0.7336	0.6845	0.6121	424338	316815	969129	9639
Bottleneck Transformer AE	Temporal	–	–	–	0.8077	0.6539	0.7059	–	–	–	–
Causal Transformer	Temporal	–	–	–	0.7099	0.5449	0.5520	–	–	–	–
Predictive LSTM	Temporal	–	–	–	0.6126	0.4472	0.5475	–	–	–	–

Tabela 4.4: CIC-IDS2017 – toate metricile raportate, pentru toate cele treisprezece combinații model/set-de-caracteristici, ordonate ca în Tabelul din §4.

Anexa 5

Verificare de Referință Trivială – Yahoo S5 (Detalii)

Această anexă completează §4.2.4 cu scorurile referinței fără etichete (norma L2 brută a ferestrei) și marja exactă, per model, față de aceasta, pe A3.

Referința obține ROC-AUC 0.508 și AP 0.348 – aproape aleator, așa cum era de așteptat pentru anomalii contextuale care sunt violări de fază, nu vârfuluri de amplitudine.

- **LSTM AE (+0.04 AP):** abia peste zgomot.
- **MLP AE (+0.16 AP):** captează implicit unele statistici privind forma ferestrei.
- **Transformare (+0.28–0.29 AP):** atenția globală asupra ferestrei de 64 de pași recuperează periodicitatea direct, o marjă mare și semnificativă.
- **Predictive LSTM (+0.47 AP):** depășește de peste două ori AP-ul referinței.

AP-ul propriu al referinței (0.348) se apropie îndeaproape de prior-ul de $\sim 33\%$ ferestre pozitive pe A3 (el însuși ridicat de estomparea etichetei la nivel de fereastră discutată în secțiunea 3.1), confirmând că referința este zgomot pur și că AP-ul brut al unui model este parțial o funcție a acestui prior umflat, nu doar a structurii învățate.

Anexa 6

Compoziția pe Clase de Atac a Setului de Test CIC-IDS2017

Tabelul 6.1 detaliază pe clase fluxurile de atac agregate din setul de test flat/context (§4); Tabelul 6.2 detaliază, analog, ferestrele de atac agregate din setul de test al pipeline-ului temporal. DoS Hulk este inclus în ambele: fluxurile/ferestrele sale rămân parte a clasei pozitive pentru orice metrică agregată, deși este exclus din detalierea pe clasă din secțiunea 4.3.2.

Clasă de atac	n (fluxuri)	% din grupul de atacuri
PortScan	159,000	36.66%
DoS Hulk	158,469	36.54%
DDoS	95,000	21.91%
DoS GoldenEye	7,500	1.73%
DoS slowloris	4,000	0.92%
FTP-Patator	4,000	0.92%
SSH-Patator	3,000	0.69%
DoS Slowhttptest	1,700	0.39%
Bot	738	0.17%
Web Attack – Brute Force	151	0.03%
Infiltration	32	0.01%
Web Attack – XSS	27	0.01%
Web Attack – SQL Injection	12	<0.01%
Heartbleed	11	<0.01%

Tabela 6.1: CIC-IDS2017 – compoziția pe clase de atac a fluxurilor de atac agregate din setul de test flat/context. DoS Hulk contribuie la fiecare metrică agregată din Tabelul 4.5, dar este exclus din detalierea pe clasă din secțiunea 4.3.2 (vezi textul).

Clasă de atac	n (ferestre)	% din grupul de atacuri
PortScan	159,160	36.42%
DoS Hulk	158,469	36.27%
DDoS	95,131	21.77%
DoS GoldenEye	7,575	1.73%
DoS slowloris	4,001	0.92%
FTP-Patator	3,966	0.91%
Bot	3,320	0.76%
SSH-Patator	2,980	0.68%
DoS Slowhttptest	1,742	0.40%
Infiltration	445	0.10%
Web Attack – Brute Force	151	0.03%
Web Attack – XSS	26	0.01%
Web Attack – SQL Injection	20	<0.01%
Heartbleed	19	<0.01%
BENIGN	1,282,611	–

Tabela 6.2: CIC-IDS2017 – compoziția pe clase de atac a ferestrelor de atac agregate din setul de test temporal (1,719,616 ferestre în total, rată de atac de 25.41%).

Anexa 7

Hărți Termice ale Erorii de Reconstrucție – Yahoo A3

Această anexă completează §5.1 cu verificarea calitativă preliminară a localizării erorii de reconstrucție. Ambele modele antrenate au fost folosite pentru reconstrucție pe fiecare fereastră anormală din setul de test A3, iar eroarea pătratică per pas temporal a fost reprezentată ca o hartă termică (câte un rând per fereastră, câte o coloană per pas temporal), pe un subeșantion aleator de 300 de ferestre.

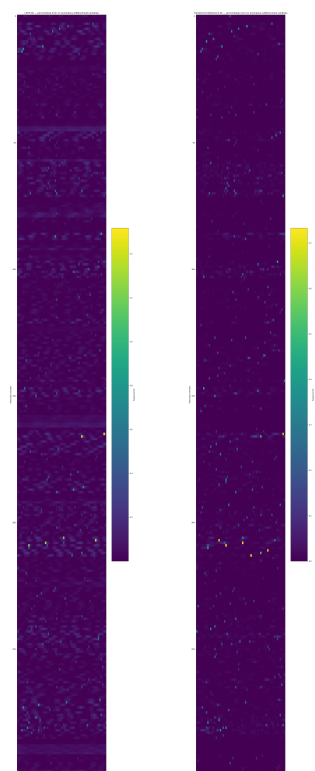


Figura 7.1: LSTM Autoencoder (stânga) vs. Transformer Bottleneck Autoencoder (dreapta) – eroarea pătratică de reconstrucție per pas temporal pe aceleași 300 de ferestre A3 anormale, eșantionate aleator (câte un rând per fereastră, aceeași ordine a rândurilor). Eroarea ridicată a LSTM AE apare mai des răspândită subțire pe toată lățimea, cu un nivel de bază ușor ridicat pe multe rânduri; eroarea Transformer-ului este vizibil mai rară și limitată la pași temporali izolați.

Comparația vizuală este în linii mari consistentă cu ipoteza inițială – eroarea Transformer-ului se concentrează în puncte mai puține și mai luminoase, în timp ce eroarea LSTM Autoencoder-ului este mai des răspândită subțire pe toată lățimea unei ferestre. Totuși, harta termică citită cu ochiul liber nu este îndeajuns pentru a confirma mecanismul: fiecare panou este normalizat pe propria sa scală de culoare, iar valorile de eroare ale LSTM Autoencoder-ului sunt, ca

magnitudine, aproximativ duble față de cele ale Transformer-ului pe tot parcursul – de unde necesitatea verificării cantitative din §5.1.

Anexa 8

Validarea Scorului Temporal IForest Sub Etichetare pe Ultimul Flux

Cifrele de AP per clasă arătate inițial folosesc o etichetă de fereastră „oricare-din-16”, adică o fereastră este pozitivă dacă oricare din cele 16 fluxuri ale sale este un atac, etichetată cu tipul de atac majoritar. Aceasta este mai permisivă decât etichetarea per flux față de care este evaluat AE+Context. Fiecare fereastră de test existentă a fost reetichetată în schimb cu eticheta reală proprie a fluxului *ultim*, iar AP-ul per clasă a fost recalculat față de modelul deja antrenat, neschimbat, și scorurile sale existente.

	AP (oricare-din-16)	AP (ultimul flux)	n (ultimul flux)
DDoS	0.9518	0.9515	95,123
DoS GoldenEye	0.6278	0.6265	7,567
DoS Slowhttptest	0.6851	0.6849	1,742
DoS slowloris	0.6091	0.6077	4,001
Heartbleed	0.5927	0.2892	11

Se observă că AP-ul rămâne practic neschimbat pentru toate cele patru clase țintă sub etichetarea mai strictă, confirmând că scorul este informativ despre statusul propriu al ultimului flux, nu un artefact al etichetării mai permisive. Heartbleed scade brusc, dar pe doar 11 fluxuri reale – întreaga sa populație de test – deci este exclus din ansamblu și tratat ca provizoriu, consistent cu tratamentul său rezervat din Capitolul 4.

Anexa 9

Diagnostic SHAP Suplimentare

Această anexă completează §5.2.1 cu doi diagnostici suplimentari pentru atribuirea SHAP a lui Temporal IForest pe ferestre DDoS și DoS slowloris.

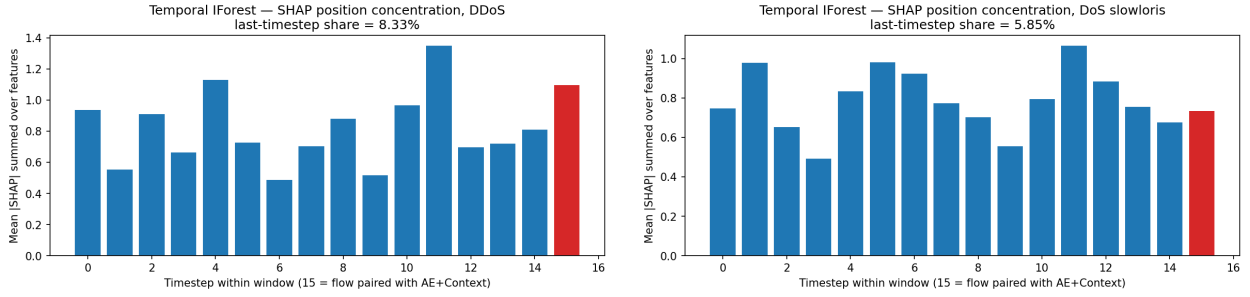


Figura 9.1: Temporal IForest – $|\text{SHAP}|$ mediu, sumat pe caracteristici, per pas temporal, ferestre DDoS (stânga) și ferestre DoS slowloris (dreapta). Ultimul pas temporal (bara roșie, 15) – fluxul combinat cu scorul lui AE+Context în ansamblul propus – nu poartă mai multă masă de atribuire decât orice altă poziție din fereastră, confirmând statistica sumară raportată în §5.2.1.

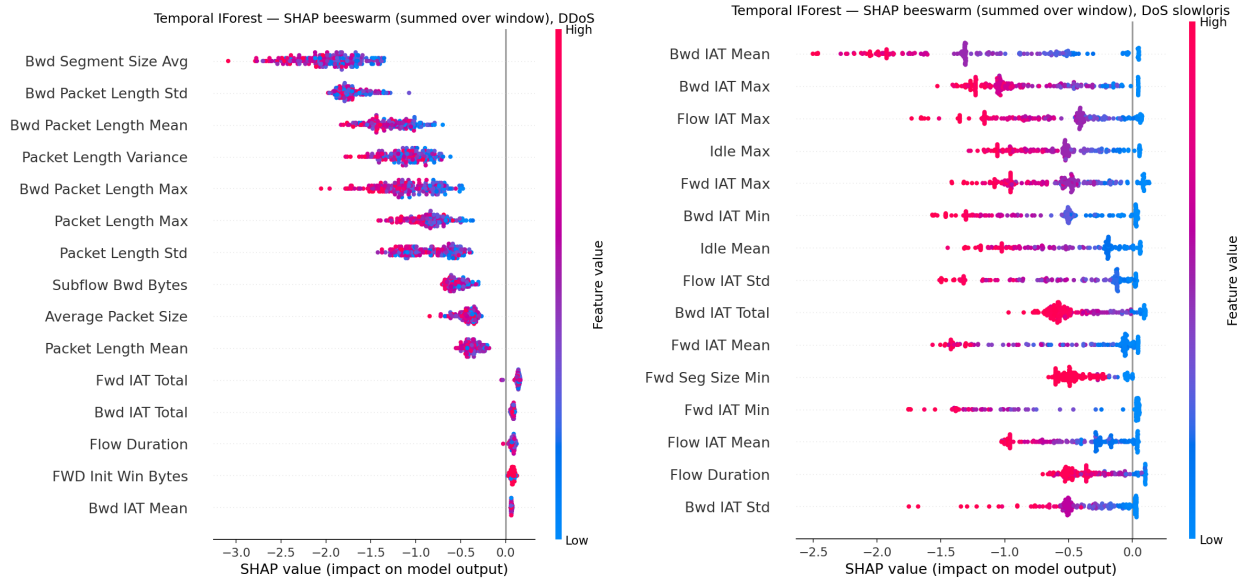


Figura 9.2: Temporal IForest – diagramă beeswarm SHAP per caracteristică, ferestre DDoS (stânga) și ferestre DoS slowloris (dreapta), colapsând axa celor 16 pași temporali prin sumarea contribuției SHAP a fiecărei caracteristici pe toată fereastră și colorând după valoarea medie a acelei caracteristici pe fereastră. DoS slowloris arată o relație monotonă clară – valori mari ale timpului între sosiri (inter-arrival time) determină un SHAP puternic negativ (anormal), valorile mici împing spre zero – consistent cu comportamentul lent, de menținere a conexiunii, după care este numit atacul. DDoS arată o relație valoare-semn mai zgomotoasă pentru caracteristicile sale de lungime a pachetelor pe direcția inversă, și o răspândire per fereastră mai largă decât sugerează, singură, imaginea de medii din Figura 5.2.

Anexa 10

Alinierea Scorurilor și Eșecul Standardizării z pentru Ansamblul AE+Context + Temporal IForest

Întrucât cele două pipeline-uri de preprocesare reordonează fluxurile diferit înainte de a-și salva propriile array-uri de test (AE+Context sortează global după marca temporală; Temporal IForest sortează per gazdă IP sursă), cele două array-uri de scoruri nu sunt aliniate index-cu-index și trebuie realiniate după identitatea fluxului, recuperând poziția originală a rândului fiecărui flux în intrarea comună, pre-sortare, a ambelor pipeline-uri.

Prima normalizare încercată, standardizarea z (z-scoring) față de media și deviația standard proprii de antrenare benignă ale fiecărui model, s-a dovedit a fi o fundătură, nu un test autentic al niciunei reguli de combinare. Scorurile z de eroare de reconstrucție ale lui AE+Context au cozi extrem de grele (maxim $z \approx 2366$ pe setul de test, cu percentila 99.9 a fluxurilor *benigne* singure deja la $z \approx 227$), în timp ce scorurile z ale lui Temporal IForest rămân strâns mărginite (maxim $z \approx 21$). Sub normalizarea z , o mână de valori extreme ale lui AE+Context domină orice combinație liniară sau maxim, indiferent de ponderea alocată lui Temporal IForest: atât regula maximului, cât și fiecare pondere de votare testată au reprodus aproape exact clasamentul propriu al lui AE+Context per clasă (de exemplu, AP-ul GoldenEye a rămas la 0.215–0.216 pe toate cele cinci ponderi, față de 0.215 al lui AE+Context singur), fără a arăta nicio influență din scorul de sine stătător mult mai puternic al lui Temporal IForest, 0.627, pe acea clasă. Acesta este un artefact de normalizare, nu o dovadă că metoda combinării nu ajută – nu testează de fapt ipoteza ansamblului.

Înlocuirea standardizării z cu normalizarea pe rang – mapând fiecare scor de test la percentila sa în distribuția scorurilor proprii de antrenare benignă ale acelui model, prin funcția de distribuție cumulativă empirică (CDF) – elimină această problemă prin construcție: fiecare scor normalizat este mărginit la $[0, 1]$ indiferent cât de extremă este valoarea brută, deci coada niciunui model nu poate domina coada celuilalt prin simpla magnitudine.

Bibliografie

- [1] Machine Learning Group – ULB, *Credit Card Fraud Detection*, <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud> [accesat: 01.09.2026].
- [2] J. Bachmann, *Credit Fraud // Dealing with Imbalanced Datasets*, <https://www.kaggle.com/code/janiobachmann/credit-fraud-dealing-with-imbalanced-datasets> [accesat: 01.09.2026].

- [3] G. Engelen, V. Rimmer, W. Joosen, „Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study”, in *2021 IEEE Security and Privacy Workshops (SPW) – Workshop on Traffic Measurements for Cybersecurity (WTMC)*, 2021, DOI: [10.1109/SPW53761.2021.00009](https://doi.org/10.1109/SPW53761.2021.00009).
- [4] M. Munir, S. A. Siddiqui, A. Dengel, S. Ahmed, „DeepAnT: A Deep Learning Approach for Unsupervised Anomaly Detection in Time Series”, in *IEEE Access* 7 (2019), pp. 1991–2005, DOI: [10.1109/ACCESS.2018.2886457](https://doi.org/10.1109/ACCESS.2018.2886457).
- [5] A. Raskovalov, N. Gabdullin, I. Androsov, „NIDS Neural Networks Using Sliding Time Window Data Processing with Trainable Activations and its Generalization Capability”, in *arXiv preprint arXiv:2410.18658* (2024), <https://arxiv.org/abs/2410.18658>.
- [6] S. Kim, K. Choi, H.-S. Choi, B. Lee, S. Yoon, „Towards a Rigorous Evaluation of Time-series Anomaly Detection”, in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, 7, 2022, pp. 7194–7201, <https://arxiv.org/abs/2109.05257>.
- [7] S. M. Lundberg, S.-I. Lee, „A Unified Approach to Interpreting Model Predictions”, in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, Long Beach, CA, USA, 2017, <https://arxiv.org/abs/1705.07874>.
- [8] SHAP developers, *SHAP (SHapley Additive exPlanations) Documentation*, <https://shap.readthedocs.io/> [accesat: 01.09.2026].
- [9] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, S.-I. Lee, „Explainable AI for Trees: From Local Explanations to Global Understanding”, in *arXiv preprint arXiv:1905.04610* (2019), <https://arxiv.org/abs/1905.04610>.
- [10] scikit-learn developers, *sklearn.svm.OneClassSVM*, <https://scikit-learn.org/stable/modules/generated/sklearn.svm.OneClassSVM.html> [accesat: 01.09.2026].
- [11] scikit-learn developers, *sklearn.ensemble.IsolationForest*, <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html> [accesat: 01.09.2026].